1.0 Project Overview

1.1 Real Life Problem

Traffic accidents are a common and persistent public safety issue worldwide, occurring daily in both urban and rural settings. According to the World Health Organization, road traffic accidents result in over a million fatalities each year, with many people experiencing both health and financial losses. In major cities, these accidents can be contributed by either:

- Human related factors such as distracted driving, speeding or impaired driving.
- Vehicle related factors such as vehicle defect and vehicle model.
- Environmental related factors such as poor weather, low visibility and road surface issues such as poorly maintained roads.

Understanding these factors is essential to reducing traffic accidents and developing effective safety measures.

1.2 Invested stakeholders

Several groups might benefit significantly from the traffic accidents data and insights drawn from understanding the primary contributory causes. These might include:

1. Government and city authorities - who might use this to identify high risk areas and improve on road infrastructure while implementing policies aimed at reducing accidents.
2. Public safety organizations - such as the Vehicle Safety Board who might use the data to aid in designing regulation and safety campaigns and enforcement strategies based on observed trends.
3. General public and drivers - who might benefit from better awareness of risk factors.
4. Vehicle manufacturers - who might leverage on insights to improve car safety features such as weather detection systems.

2.0 Business Understanding

2.1 Business Problem

Large cities such as the City of Chicago experience frequent traffic accidents often. To address this challenge, there is need for a predictive model that can identify **primary contributory cause of car accidents** using key influencing factors including:

1. Human Factors such as sex and age.
2. Vehicle Factors such as vehicle make.
3. Environmental Factors such as weather patterns.

4. Temporal Factors such as Date and Time of the Accidents.

5. Road Design Characteristics such as road defects and road surface conditions.

Such a model would support data driven decision making aimed at reducing accident rates and improving overall road safety.

2.2 Business Questions

The project seeks to answer the following business questions:

1. What are the most common primary contributory causes of traffic accidents?

2. How do human characteristics contribute to accident occurrence?

3. How do vehicle characteristics contribute to accident occurrence?

4. How do environmental conditions such as weather influence accident occurrence?

5. Are there specific times when accidents are more likely to occur?

6. How does road design impact the likelihood of accidents?

7. Can we accurately predict the primary cause of an accident given the human, vehicle, environmental, temporal and road design characteristics?

2.3 Business Objectives

The main objectives of this project are to:

1. Develop a predictive model that can classify the primary contributory cause of traffic accidents.

2. Identify key factors (Human, Vehicle, Environmental, Temporal) that significantly influence accident occurrence.

3.0 Data Understanding

3.1 Data Description and Sources

- This project utilized 3 datasets that contain information on the accident occurence, characteristics of people involved as well as the vehicle characteristics. These are:

1. City of Chicago Accident Dataset - sourced from https://data.cityofchicago.org/Transportation/Traffic-Crashes-Crashes/85ca-t3if. This dataset has 1.04 million rows and 48 columns. Each row is a traffic crash with the row identifier as the CRASH_RECORD_ID.
2. Vehicle Accident Dataset sourced from https://data.cityofchicago.org/Transportation/Traffic-Crashes-Vehicles/68nd-jvt3. It has 2.28 million rows and 29 columns. Each row is a person with the row identifier as the PERSON_ID. The dataset also has CRASH_RECORD_ID.

3. Driver/Passenger Accident Dataset sourced from https://data.cityofchicago.org/Transportation/Traffic-Crashes-People/u6pd-qa9d. It has 2.12 million rows with 71 columns. Each row is a vehicle with the row identifier as the CRASH_UNIT_ID. This dataset however also has a CRASH_RECORD_ID.

Note: Our goal is to merge the three datasets using the common CRASH_RECORD_ID, creating a unified dataset that combines all relevant features for our analysis.

From the merger, we aim to use the following columns from the following datasets:

1. City of Chicago Dataset:

- Weather condition - Weather condition at time of crash.
- Lighting condition - Light condition at time of crash.
- Road defects - Road defects, as determined by reporting officer.
- Crash type - A general severity classification for the crash. Can be either Injury and/or Tow Due to Crash or No Injury / Drive Away.
- Primary Contributory Cause - The factor which was most significant in causing the crash.
- Crash hour - The hour of the day component of CRASH_DATE.
- Crash day - The day of the week component of CRASH_DATE.
- Crash month - The month component of CRASH_DATE.

1. Vehicle Dataset:

- Number of passengers - Number of passengers in the vehicle. The driver is not included.
- Make - The make (brand) of the vehicle.
- Vehicle Use - The normal use of the vehicle.
- Vehicle Defect - Defect the vehicle had.
- Exceeds speed limit - Whether vehicle exceeded speed limit.

1. Passenger Dataset:

- Person type - Type of roadway user involved in crash.
- Sex - Gender of person involved in crash.
- Age - Age of person involved in crash.
- Driver Vision - What, if any, objects obscured the driver's vision at time of crash.
- Physical Condition - Driver's apparent physical condition at time of crash.
- Driver Action - Driver action that contributed to the crash.
- Pedpedal Action - Action of pedestrian or cyclist at the time of crash.

3.2 Target Variable

- Our target variable is: Primary contributory cause which is a multiclass classification problem. There are 40 values of the aforementioned target variable and we aim to predict the most common primary contributory causes based on our feature variables.

3.3 Feature Variables

- All the other aforementioned variables other than primary contributory cause will act as our feature variables. They will be specifically selected following a merger of the 3 Datasets.

3.4 Challenge with large samples of rows

Following the merger, we expect 2 challenges:

1. Since a single crash may involve more than just the driver, each CRASH_RECORD_ID could correspond to multiple people. To simplify the analysis, we will focus on drivers, examining each crash event from the perspective of the drivers involved.
2. A single crash can involve multiple vehicles, so if a crash involves two cars with one person in each, it would result in at least two rows—one for each individual involved in the same crash.
3. The dataset contains millions of rows, which may exceed available computing resources. To manage this, we will limit our analysis to approximately 500,000 randomly selected samples after data cleaning, which will also help reduce the dataset size naturally.

4.0 Data Preparation and Cleaning

4.1 Data Preparation

- In this section we will load the data, select the columns we want to work with and limit it only to drivers

In [1]:
```
# We will start by importing the necessary libraries for data cleaning and visualiza

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline

# Libraries for data model preperation
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
import joblib
from sklearn.compose import ColumnTransformer

# Libraries for Machine Learning
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
import tensorflow as tf
from tensorflow.keras import layers, models, callbacks
from tensorflow import keras
from sklearn.neural_network import MLPClassifier

# Libraries for evaluation
from sklearn.metrics import accuracy_score, classification_report

# Libraries for hyper parameter tuning
from sklearn.model_selection import GridSearchCV
```

```
/Users/administrator/Library/Python/3.9/lib/python/site-packages/urllib3/__init__.py:
35: NotOpenSSLWarning: urllib3 v2 only supports OpenSSL 1.1.1+, currently the 'ssl' m
odule is compiled with 'LibreSSL 2.8.3'. See: https://github.com/urllib3/urllib3/issu
es/3020
  warnings.warn(
```

In [2]:
```
# Loading the 3 Datasets and saving them under different variable names. Starting wi
```

```
accidents_log = pd.read_csv("accidents.csv", low_memory = False) # Low memory helps
accidents_log.head()
```

Out[2]:

	CRASH_RECORD_ID	CRASH_DATE_EST_I	CRASH_DATE	POSTED_SPE
0	c4658ba1425251e03a299caf165125eba8af03229e2238...	NaN	03/24/2026 11:10:00 PM	
1	169bb4297ce8f9de0b432baae8e5c61fccff5f877be0e0...	NaN	03/24/2026 10:45:00 PM	
2	ebc0228a98d221f831a0f232dc0c310c2d15d47e1a976b...	NaN	03/24/2026 10:25:00 PM	
3	4ca882872708f63d8e3f3d30f0b4cd885132273e9b7708...	NaN	03/24/2026 10:18:00 PM	
4	f8c941528e1bdd2a112d6c25ea37e289e92e724498b198...	NaN	03/24/2026 10:09:00 PM	

5 rows × 48 columns

In [3]:

```
# Loading the passenger data and checking our data
passenger_log = pd.read_csv("passenger.csv", low_memory = False)
passenger_log.head()
```

Out[3]:

	PERSON_ID	PERSON_TYPE	CRASH_RECORD_ID	VEHICLE_ID	CRASI
0	O2271292	DRIVER	169bb4297ce8f9de0b432baae8e5c61fccff5f877be0e0...	2165998.0	03/ 10:4
1	P499550	PASSENGER	169bb4297ce8f9de0b432baae8e5c61fccff5f877be0e0...	2165998.0	03/ 10:4
2	O2271266	DRIVER	ebc0228a98d221f831a0f232dc0c310c2d15d47e1a976b...	2165976.0	03/ 10:2
3	O2271267	DRIVER	ebc0228a98d221f831a0f232dc0c310c2d15d47e1a976b...	2165981.0	03/ 10:2
4	O2271268	DRIVER	4ca882872708f63d8e3f3d30f0b4cd885132273e9b7708...	2165979.0	03/ 10:1

5 rows × 29 columns

In [4]:

```
# Loading the vehicle data and checking our data
vehicle_log = pd.read_csv("vehicles.csv", low_memory = False)
vehicle_log.head()
```

Out[4]:

	CRASH_UNIT_ID	CRASH_RECORD_ID	CRASH_DATE	UNIT_NO	UNI
0	2271287	c4658ba1425251e03a299caf165125eba8af03229e2238...	03/24/2026 11:10:00 PM	1	DRIV
1	2271292	169bb4297ce8f9de0b432baae8e5c61fccff5f877be0e0...	03/24/2026 10:45:00 PM	1	

	CRASH_UNIT_ID	CRASH_RECORD_ID	CRASH_DATE	UNIT_NO	UNI
2	2271293	169bb4297ce8f9de0b432baae8e5c61fccff5f877be0e0...	03/24/2026 10:45:00 PM	2	I
3	2271266	ebc0228a98d221f831a0f232dc0c310c2d15d47e1a976b...	03/24/2026 10:25:00 PM	1	
4	2271267	ebc0228a98d221f831a0f232dc0c310c2d15d47e1a976b...	03/24/2026 10:25:00 PM	2	

5 rows × 71 columns

In [5]:
```
# Checking the different columns in each dataset
accidents_log.columns, passenger_log.columns, vehicle_log.columns
```

Out[5]:
```
(Index(['CRASH_RECORD_ID', 'CRASH_DATE_EST_I', 'CRASH_DATE',
        'POSTED_SPEED_LIMIT', 'TRAFFIC_CONTROL_DEVICE', 'DEVICE_CONDITION',
        'WEATHER_CONDITION', 'LIGHTING_CONDITION', 'FIRST_CRASH_TYPE',
        'TRAFFICWAY_TYPE', 'LANE_CNT', 'ALIGNMENT', 'ROADWAY_SURFACE_COND',
        'ROAD_DEFECT', 'REPORT_TYPE', 'CRASH_TYPE', 'INTERSECTION_RELATED_I',
        'NOT_RIGHT_OF_WAY_I', 'HIT_AND_RUN_I', 'DAMAGE', 'DATE_POLICE_NOTIFIED',
        'PRIM_CONTRIBUTORY_CAUSE', 'SEC_CONTRIBUTORY_CAUSE', 'STREET_NO',
        'STREET_DIRECTION', 'STREET_NAME', 'BEAT_OF_OCCURRENCE',
        'PHOTOS_TAKEN_I', 'STATEMENTS_TAKEN_I', 'DOORING_I', 'WORK_ZONE_I',
        'WORK_ZONE_TYPE', 'WORKERS_PRESENT_I', 'NUM_UNITS',
        'MOST_SEVERE_INJURY', 'INJURIES_TOTAL', 'INJURIES_FATAL',
        'INJURIES_INCAPACITATING', 'INJURIES_NON_INCAPACITATING',
        'INJURIES_REPORTED_NOT_EVIDENT', 'INJURIES_NO_INDICATION',
        'INJURIES_UNKNOWN', 'CRASH_HOUR', 'CRASH_DAY_OF_WEEK', 'CRASH_MONTH',
        'LATITUDE', 'LONGITUDE', 'LOCATION'],
       dtype='object'),
 Index(['PERSON_ID', 'PERSON_TYPE', 'CRASH_RECORD_ID', 'VEHICLE_ID',
        'CRASH_DATE', 'SEAT_NO', 'CITY', 'STATE', 'ZIPCODE', 'SEX', 'AGE',
        'DRIVERS_LICENSE_STATE', 'DRIVERS_LICENSE_CLASS', 'SAFETY_EQUIPMENT',
        'AIRBAG_DEPLOYED', 'EJECTION', 'INJURY_CLASSIFICATION', 'HOSPITAL',
        'EMS_AGENCY', 'EMS_RUN_NO', 'DRIVER_ACTION', 'DRIVER_VISION',
        'PHYSICAL_CONDITION', 'PEDPEDAL_ACTION', 'PEDPEDAL_VISIBILITY',
        'PEDPEDAL_LOCATION', 'BAC_RESULT', 'BAC_RESULT VALUE',
        'CELL_PHONE_USE'],
       dtype='object'),
 Index(['CRASH_UNIT_ID', 'CRASH_RECORD_ID', 'CRASH_DATE', 'UNIT_NO',
        'UNIT_TYPE', 'NUM_PASSENGERS', 'VEHICLE_ID', 'CMRC_VEH_I', 'MAKE',
        'MODEL', 'LIC_PLATE_STATE', 'VEHICLE_YEAR', 'VEHICLE_DEFECT',
        'VEHICLE_TYPE', 'VEHICLE_USE', 'TRAVEL_DIRECTION', 'MANEUVER',
        'TOWED_I', 'FIRE_I', 'OCCUPANT_CNT', 'EXCEED_SPEED_LIMIT_I', 'TOWED_BY',
        'TOWED_TO', 'AREA_00_I', 'AREA_01_I', 'AREA_02_I', 'AREA_03_I',
        'AREA_04_I', 'AREA_05_I', 'AREA_06_I', 'AREA_07_I', 'AREA_08_I',
        'AREA_09_I', 'AREA_10_I', 'AREA_11_I', 'AREA_12_I', 'AREA_99_I',
        'FIRST_CONTACT_POINT', 'CMV_ID', 'USDOT_NO', 'CCMC_NO', 'ILCC_NO',
        'COMMERCIAL_SRC', 'GVWR', 'CARRIER_NAME', 'CARRIER_STATE',
        'CARRIER_CITY', 'HAZMAT_PLACARDS_I', 'HAZMAT_NAME', 'UN_NO',
        'HAZMAT_PRESENT_I', 'HAZMAT_REPORT_I', 'HAZMAT_REPORT_NO',
        'MCS_REPORT_I', 'MCS_REPORT_NO', 'HAZMAT_VIO_CAUSE_CRASH_I',
        'MCS_VIO_CAUSE_CRASH_I', 'IDOT_PERMIT_NO', 'WIDE_LOAD_I',
        'TRAILER1_WIDTH', 'TRAILER2_WIDTH', 'TRAILER1_LENGTH',
        'TRAILER2_LENGTH', 'TOTAL_VEHICLE_LENGTH', 'AXLE_CNT', 'VEHICLE_CONFIG',
        'CARGO_BODY_TYPE', 'LOAD_TYPE', 'HAZMAT_OUT_OF_SERVICE_I',
        'MCS_OUT_OF_SERVICE_I', 'HAZMAT_CLASS'],
       dtype='object'))
```

In [6]:
```
# Select ONLY the specific columns we outlined in Section 3.1
crash_cols = ['CRASH_RECORD_ID', 'WEATHER_CONDITION', 'LIGHTING_CONDITION', 'ROAD_DE
              'CRASH_TYPE', 'PRIM_CONTRIBUTORY_CAUSE', 'CRASH_HOUR', 'CRASH_DAY_OF_W

vehicle_cols = ['CRASH_RECORD_ID', 'OCCUPANT_CNT', 'MAKE',
                'VEHICLE_USE', 'VEHICLE_DEFECT', 'EXCEED_SPEED_LIMIT_I']

passenger_cols = ['CRASH_RECORD_ID', 'PERSON_TYPE', 'SEX', 'AGE', 'DRIVER_VISION',
                  'PHYSICAL_CONDITION', 'DRIVER_ACTION', 'PEDPEDAL_ACTION']

# Subset the dataframes to drop the 100+ columns we don't need
accidents_sub = accidents_log[crash_cols]
vehicle_sub = vehicle_log[vehicle_cols]
passenger_sub = passenger_log[passenger_cols]
```

In [7]:
```
# Going forward, we want to focus our dataset purely on factors that affect drivers
# Of the 3 datasets, we can specify drivers only in the passenger_log data as this h
# The other datasets do not have 'persons' column to specify the person involved in
# However, after specifying the drivers and merging the datasets, we expect to have

drivers_only = passenger_sub[passenger_sub['PERSON_TYPE'] == 'DRIVER'].copy()
```

In [8]:
```
# We now want to merge the 3 datasets; accidents, passenger, and vehicles csv datase
# From the above, we notice that the 3 datasets share the 'CRASH_RECORD_ID' column. I
# We will first merge accidents_sub with vehicles_unique

merged_accidents_vehicles = pd.merge(
    accidents_sub,
    vehicle_sub,
    how = 'left',
    on = 'CRASH_RECORD_ID'
)

# we will then merge the combined merged_accidents_vehicles with drivers_unique to h
merged_df = pd.merge(
    drivers_only,
    merged_accidents_vehicles,
    how = 'left',
    on = 'CRASH_RECORD_ID'
)

# Check the merged_df
merged_df.head()
```

Out[8]:

	CRASH_RECORD_ID	PERSON_TYPE	SEX	AGE	DRIVER_VISION	PH
0	169bb4297ce8f9de0b432baae8e5c61fccff5f877be0e0...	DRIVER	F	32.0	NOT OBSCURED	
1	169bb4297ce8f9de0b432baae8e5c61fccff5f877be0e0...	DRIVER	F	32.0	NOT OBSCURED	
2	ebc0228a98d221f831a0f232dc0c310c2d15d47e1a976b...	DRIVER	M	40.0	NOT OBSCURED	
3	ebc0228a98d221f831a0f232dc0c310c2d15d47e1a976b...	DRIVER	M	40.0	NOT OBSCURED	
4	ebc0228a98d221f831a0f232dc0c310c2d15d47e1a976b...	DRIVER	M	50.0	NOT OBSCURED	

5 rows × 21 columns

In [9]:
```
# Checking the length of the merged datasets
len(merged_df) # we are dealing with 1,192,524  entries
```

Out[9]: 1192524

In [10]:
```
# checking the number of columns
len(merged_df.columns) # we have a total of 21 columns
```

Out[10]: 21

In [11]:
```
merged_df.head()
```

Out[11]:

	CRASH_RECORD_ID	PERSON_TYPE	SEX	AGE	DRIVER_VISION	PH
0	169bb4297ce8f9de0b432baae8e5c61fccff5f877be0e0...	DRIVER	F	32.0	NOT OBSCURED	
1	169bb4297ce8f9de0b432baae8e5c61fccff5f877be0e0...	DRIVER	F	32.0	NOT OBSCURED	
2	ebc0228a98d221f831a0f232dc0c310c2d15d47e1a976b...	DRIVER	M	40.0	NOT OBSCURED	
3	ebc0228a98d221f831a0f232dc0c310c2d15d47e1a976b...	DRIVER	M	40.0	NOT OBSCURED	
4	ebc0228a98d221f831a0f232dc0c310c2d15d47e1a976b...	DRIVER	M	50.0	NOT OBSCURED	

5 rows × 21 columns

In [12]:
```
 #Save the clean dataset into our folder
merged_df.to_csv('chicago_crashes_cleaned.csv', index=False)
```

4.2 Data Cleaning

- In this section we will explore the datatypes, look for missing values and fill them.
- We will not drop duplicates because we are assuming that each CRASH_RECORD_ID is shared between 2 drivers; the driver who caused the accident and the one affected. We will therefore treat each driver row as an independent accident.

In [13]:
```
# Exploring the datatypes

merged_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1192524 entries, 0 to 1192523
Data columns (total 21 columns):
 #   Column                  Non-Null Count    Dtype
---  ------                  --------------    -----
 0   CRASH_RECORD_ID         1192524 non-null  object
 1   PERSON_TYPE             1192524 non-null  object
 2   SEX                     1192524 non-null  object
 3   AGE                     875804 non-null   float64
```

```
 4   DRIVER_VISION            1192524 non-null  object
 5   PHYSICAL_CONDITION       1192524 non-null  object
 6   DRIVER_ACTION            1192524 non-null  object
 7   PEDPEDAL_ACTION          0 non-null        object
 8   WEATHER_CONDITION        1192524 non-null  object
 9   LIGHTING_CONDITION       1192524 non-null  object
 10  ROAD_DEFECT              1192524 non-null  object
 11  CRASH_TYPE               1192524 non-null  object
 12  PRIM_CONTRIBUTORY_CAUSE  1192524 non-null  object
 13  CRASH_HOUR               1192524 non-null  int64
 14  CRASH_DAY_OF_WEEK        1192524 non-null  int64
 15  CRASH_MONTH              1192524 non-null  int64
 16  OCCUPANT_CNT             1174243 non-null  float64
 17  MAKE                     1174243 non-null  object
 18  VEHICLE_USE              1174243 non-null  object
 19  VEHICLE_DEFECT           1174243 non-null  object
 20  EXCEED_SPEED_LIMIT_I     8 non-null        object
dtypes: float64(2), int64(3), object(16)
memory usage: 191.1+ MB
```

Of the 21 variables, 5 are numerical: CRASH_HOUR, CRASH_DAY_OF_WEEK, CRASH_MONTH, OCCUPANT_COUNT, AGE.

The remaining variables are all of the string datatype.

In [14]:
```
# Dropping the column 'CRASH_RECORD_ID' as it is not needed for our analysis
merged_df.drop(columns=['CRASH_RECORD_ID'], inplace=True)
```

In [15]:
```
# we will now check for missing data
merged_df.isna().sum()
```

Out[15]:
```
PERSON_TYPE                     0
SEX                             0
AGE                        316720
DRIVER_VISION                   0
PHYSICAL_CONDITION              0
DRIVER_ACTION                   0
PEDPEDAL_ACTION           1192524
WEATHER_CONDITION               0
LIGHTING_CONDITION              0
ROAD_DEFECT                     0
CRASH_TYPE                      0
PRIM_CONTRIBUTORY_CAUSE         0
CRASH_HOUR                      0
CRASH_DAY_OF_WEEK               0
CRASH_MONTH                     0
OCCUPANT_CNT                18281
MAKE                        18281
VEHICLE_USE                 18281
VEHICLE_DEFECT              18281
EXCEED_SPEED_LIMIT_I      1192516
dtype: int64
```

The following variables had missing values:

- AGE 316720
- PEDPEDAL_ACTION 1192524
- OCCUPANT_CNT 18281
- MAKE 18281
- VEHICLE_USE 18281
- VEHICLE_DEFECT 18281
- EXCEED_SPEED_LIMIT_I 1192516

In [16]:
```
# Given that we had 1,192,524 entries and the column with the highest missing values

merged_df.drop(columns=['EXCEED_SPEED_LIMIT_I','PEDPEDAL_ACTION'],inplace=True)

# Confirming that we have deopped the 2 columns
merged_df.columns   # we have dropped the 2 columns
```

Out[16]:
```
Index(['PERSON_TYPE', 'SEX', 'AGE', 'DRIVER_VISION', 'PHYSICAL_CONDITION',
       'DRIVER_ACTION', 'WEATHER_CONDITION', 'LIGHTING_CONDITION',
       'ROAD_DEFECT', 'CRASH_TYPE', 'PRIM_CONTRIBUTORY_CAUSE', 'CRASH_HOUR',
       'CRASH_DAY_OF_WEEK', 'CRASH_MONTH', 'OCCUPANT_CNT', 'MAKE',
       'VEHICLE_USE', 'VEHICLE_DEFECT'],
      dtype='object')
```

In [17]:
```
# Of the remaining variables, the one with the highest missing values is age with 31
# This constitiutes 26.6% of the total data. Filling in these values with either mea
# We therefore opt to drop these missing values completely.

merged_df.dropna(inplace=True)

#confirming that we have dropped all missing values
merged_df.isna().sum()  #We have dropped all missing values
```

Out[17]:
```
PERSON_TYPE                0
SEX                        0
AGE                        0
DRIVER_VISION              0
PHYSICAL_CONDITION         0
DRIVER_ACTION              0
WEATHER_CONDITION          0
LIGHTING_CONDITION         0
ROAD_DEFECT                0
CRASH_TYPE                 0
PRIM_CONTRIBUTORY_CAUSE    0
CRASH_HOUR                 0
CRASH_DAY_OF_WEEK          0
CRASH_MONTH                0
OCCUPANT_CNT               0
MAKE                       0
VEHICLE_USE                0
VEHICLE_DEFECT             0
dtype: int64
```

In [18]:
```
# A quick scan of all the variables noted inconsistencies with the age variable
pd.cut(merged_df['AGE'], bins=10).value_counts() # Our age variable has a lot of inc

# We will now exclude the entries with negative age values and then recheck the age
merged_df = merged_df[merged_df['AGE'] >= 1]


#Confirming that all negative age values have been dropped
pd.cut(merged_df['AGE'], bins=10).value_counts() # We have successfully dropped all
```

Out[18]:
```
AGE
(22.8, 33.7]     259792
(33.7, 44.6]     203605
(44.6, 55.5]     147259
(55.5, 66.4]     104247
(11.9, 22.8]      82895
(66.4, 77.3]      47329
(77.3, 88.2]      11599
(88.2, 99.1]        977
(0.891, 11.9]       814
(99.1, 110.0]        33
Name: count, dtype: int64
```

In [19]:
```
# Confirming the new number of tatal entries
len(merged_df)      #We are now working with 858,550 total complete entries.
```

Out[19]:
```
858550
```

In [20]:
```
 #Save the final cleaned dataset into our folder
merged_df.to_csv('chicago_crashes_final_dataset_cleaned.csv', index=False)
```

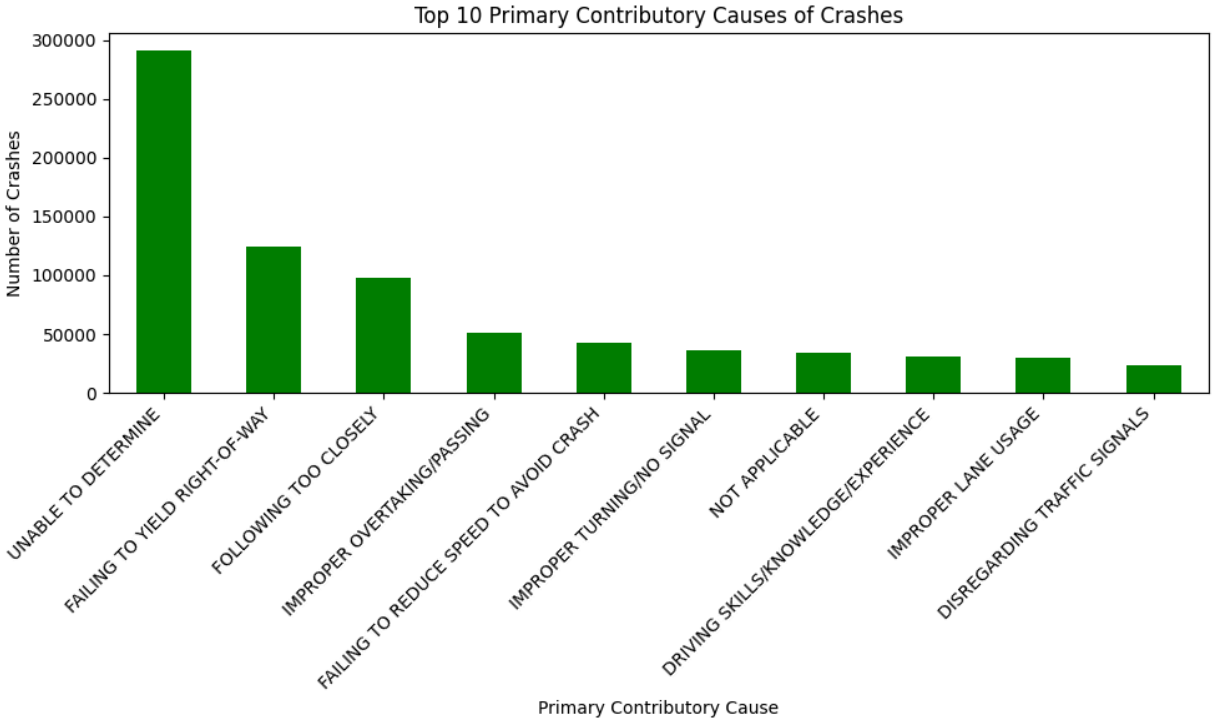
5.0 Exploratory Data Analysis

5.1 The most common primary contributory causes of traffic accidents

In [21]:
```
merged_df['PRIM_CONTRIBUTORY_CAUSE'].value_counts().head(10)
```

Out[21]:
```
PRIM_CONTRIBUTORY_CAUSE
UNABLE TO DETERMINE                          291352
FAILING TO YIELD RIGHT-OF-WAY                124226
FOLLOWING TOO CLOSELY                         98174
IMPROPER OVERTAKING/PASSING                   50970
FAILING TO REDUCE SPEED TO AVOID CRASH        42900
IMPROPER TURNING/NO SIGNAL                    36264
NOT APPLICABLE                                33996
DRIVING SKILLS/KNOWLEDGE/EXPERIENCE           30640
IMPROPER LANE USAGE                           30163
DISREGARDING TRAFFIC SIGNALS                  23579
Name: count, dtype: int64
```

In [22]:
```
#illustrating the top 10 primary contributory causes of crashes in a bar chart
fig, ax =plt.subplots(figsize=(10, 6))
cause_counts = merged_df['PRIM_CONTRIBUTORY_CAUSE'].value_counts().head(10)
cause_counts.plot(kind='bar', ax=ax, color='green')
ax.set_xlabel('Primary Contributory Cause')
ax.set_ylabel('Number of Crashes')
ax.set_title('Top 10 Primary Contributory Causes of Crashes')
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()
```



5.2 - 5.6 How do characteristics contribute to accident occurence?

In this part of the EDA we will summarize the impact of:

- Human characteristics
- Environmental conditions
- Vehicle characteristics
- Temporal factors
- Road design factors

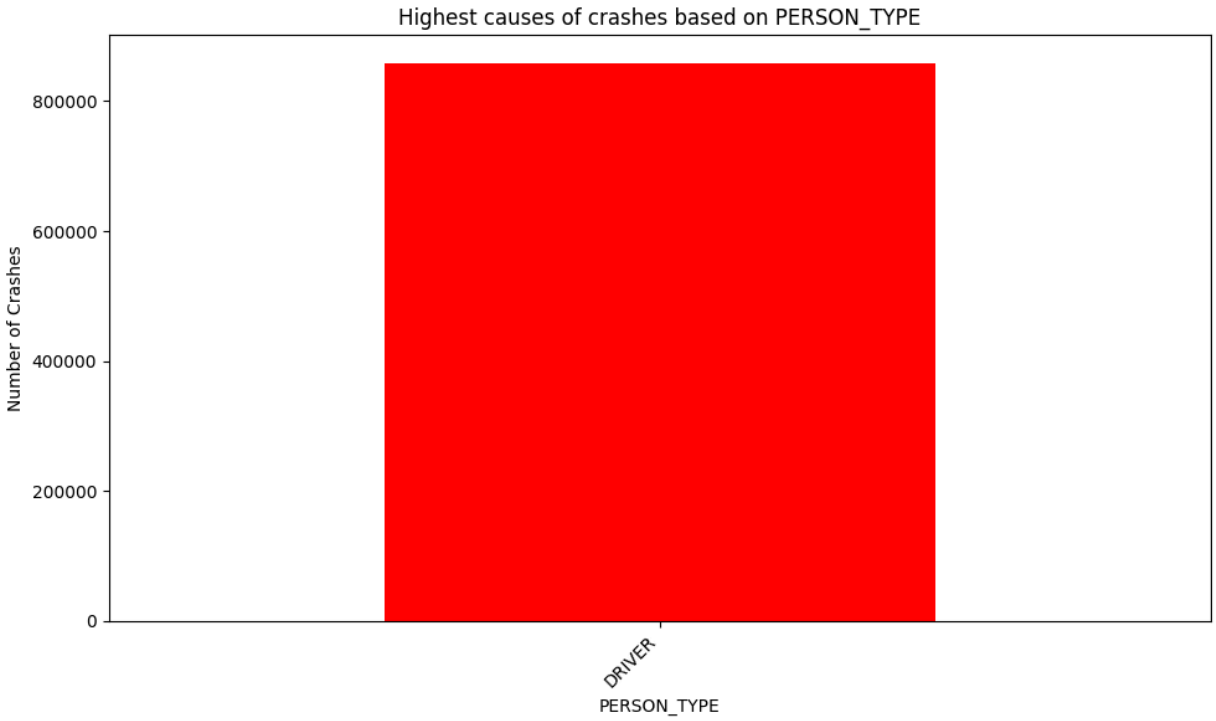
In [23]:
```
# Rechecking the data columns
merged_df.columns
```

Out[23]:
```
Index(['PERSON_TYPE', 'SEX', 'AGE', 'DRIVER_VISION', 'PHYSICAL_CONDITION',
       'DRIVER_ACTION', 'WEATHER_CONDITION', 'LIGHTING_CONDITION',
       'ROAD_DEFECT', 'CRASH_TYPE', 'PRIM_CONTRIBUTORY_CAUSE', 'CRASH_HOUR',
       'CRASH_DAY_OF_WEEK', 'CRASH_MONTH', 'OCCUPANT_CNT', 'MAKE',
       'VEHICLE_USE', 'VEHICLE_DEFECT'],
      dtype='object')
```

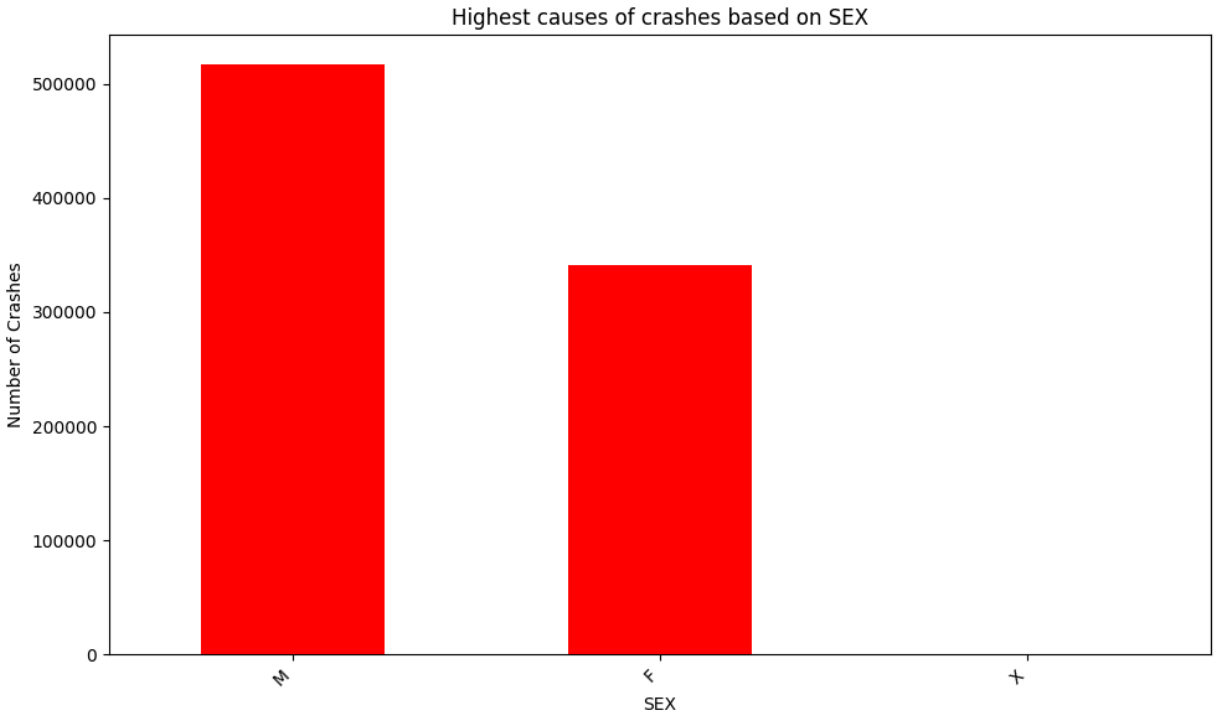
In [24]:
```
# Summarizing the impact of various categorical factors on accident occurence
for column in merged_df.columns:
    if pd.api.types.is_object_dtype(merged_df[column]):
        print(f"{column} is categorical with value counts:\n{merged_df[column].value
        # Plotting the categorical variables in a bar chart
        plt.figure(figsize=(10, 6))
        merged_df[column].value_counts().head(5).plot(kind='bar', color='red')
        plt.title(f'Highest causes of crashes based on {column}')
        plt.xlabel(column)
        plt.ylabel('Number of Crashes')
        plt.xticks(rotation=45, ha='right')
        plt.tight_layout()
        plt.show()
```

```
PERSON_TYPE is categorical with value counts:
PERSON_TYPE
DRIVER    858550
```

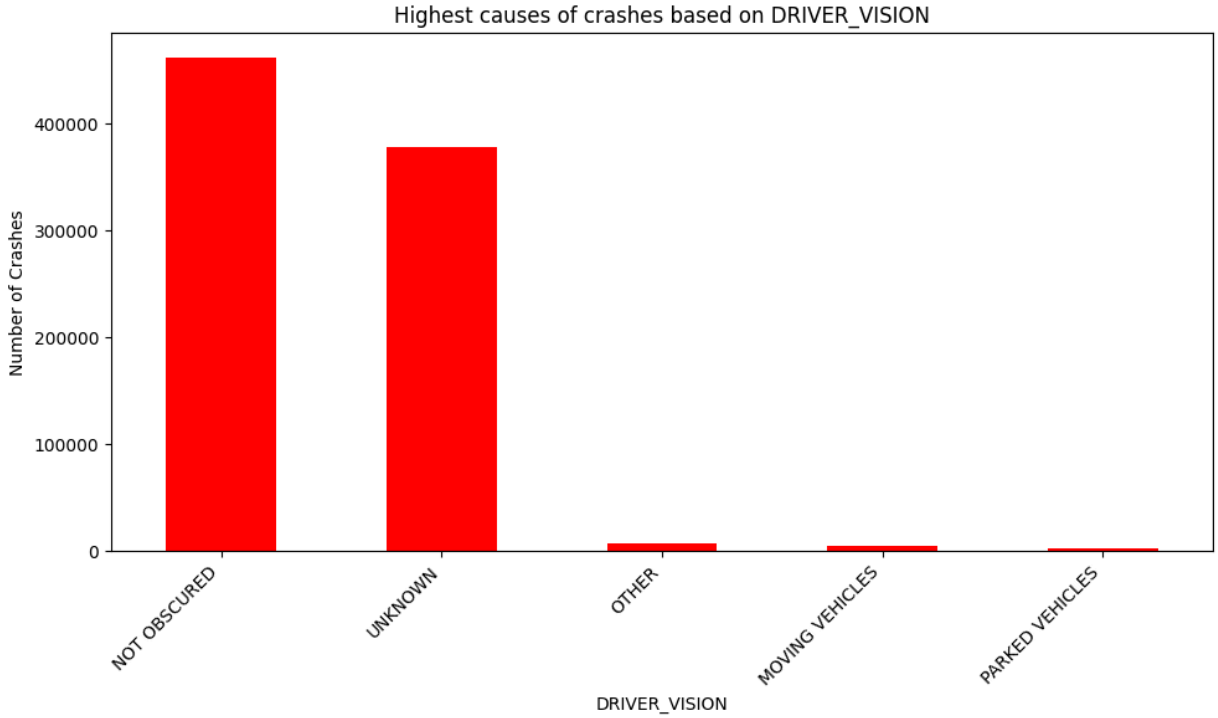
```
Name: count, dtype: int64
```



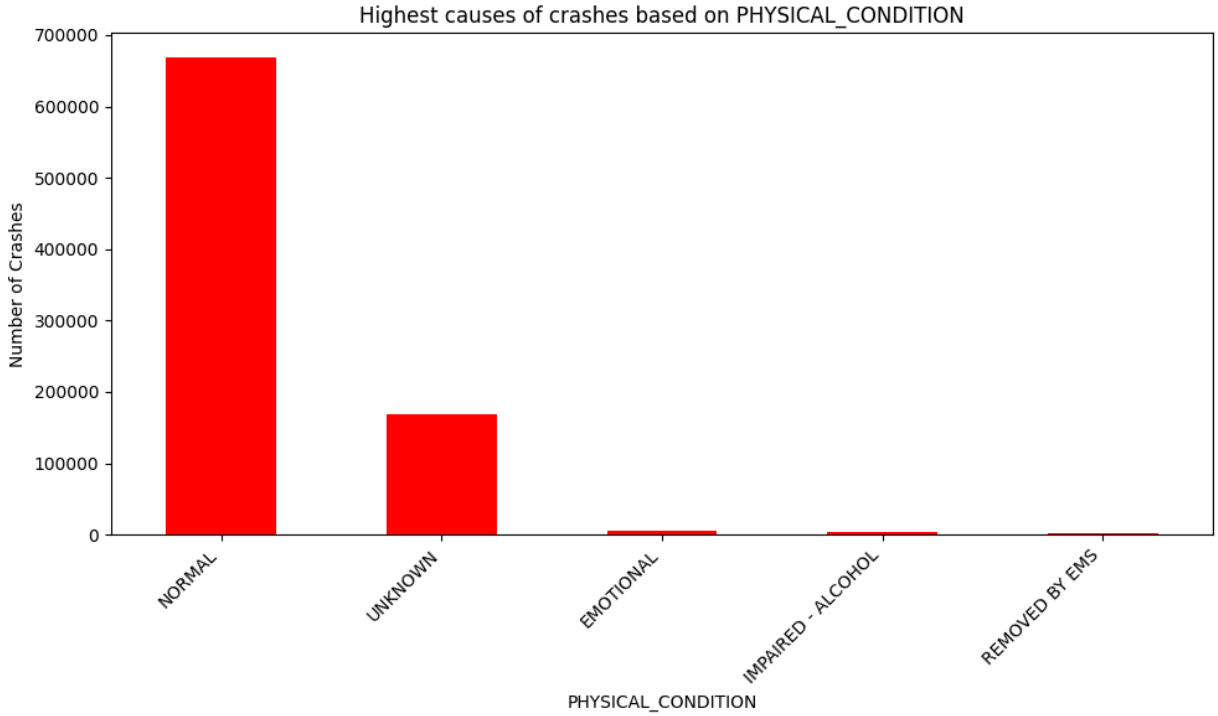

```
SEX is categorical with value counts:
SEX
M    516798
F    341029
X       723
Name: count, dtype: int64
```



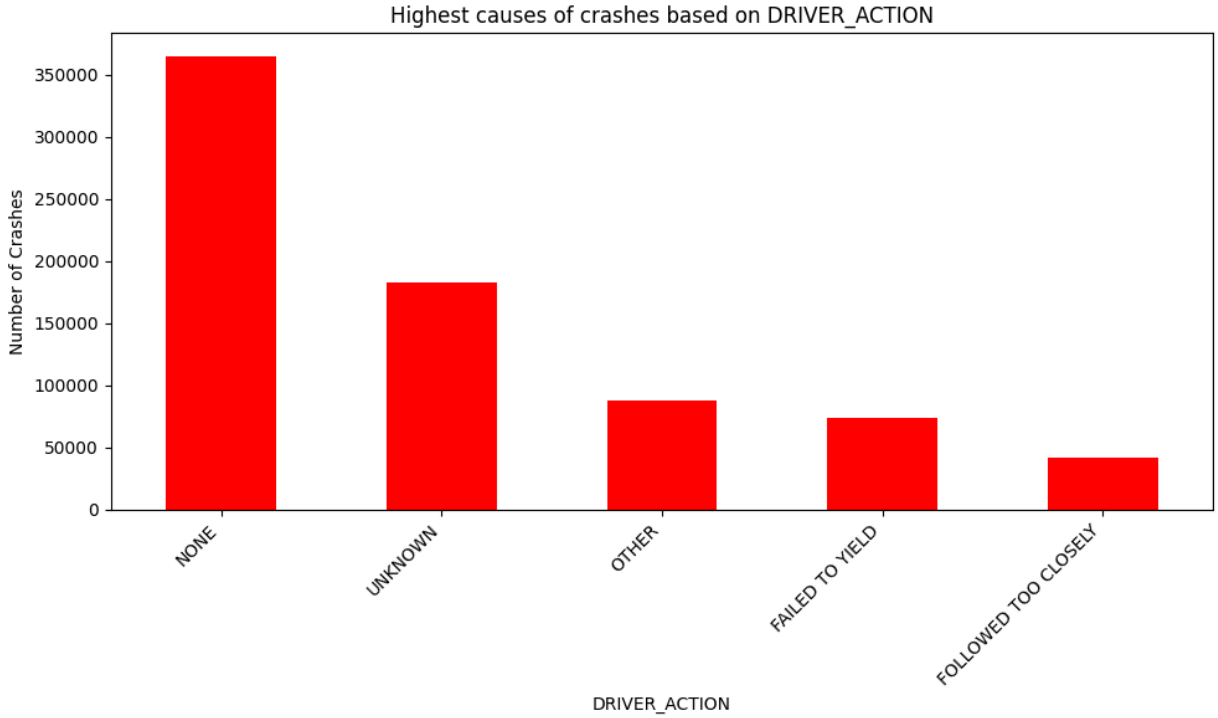

```
DRIVER_VISION is categorical with value counts:
DRIVER_VISION
NOT OBSCURED       461352
UNKNOWN            378365
OTHER                7283
MOVING VEHICLES      4751
PARKED VEHICLES      2920
Name: count, dtype: int64
```



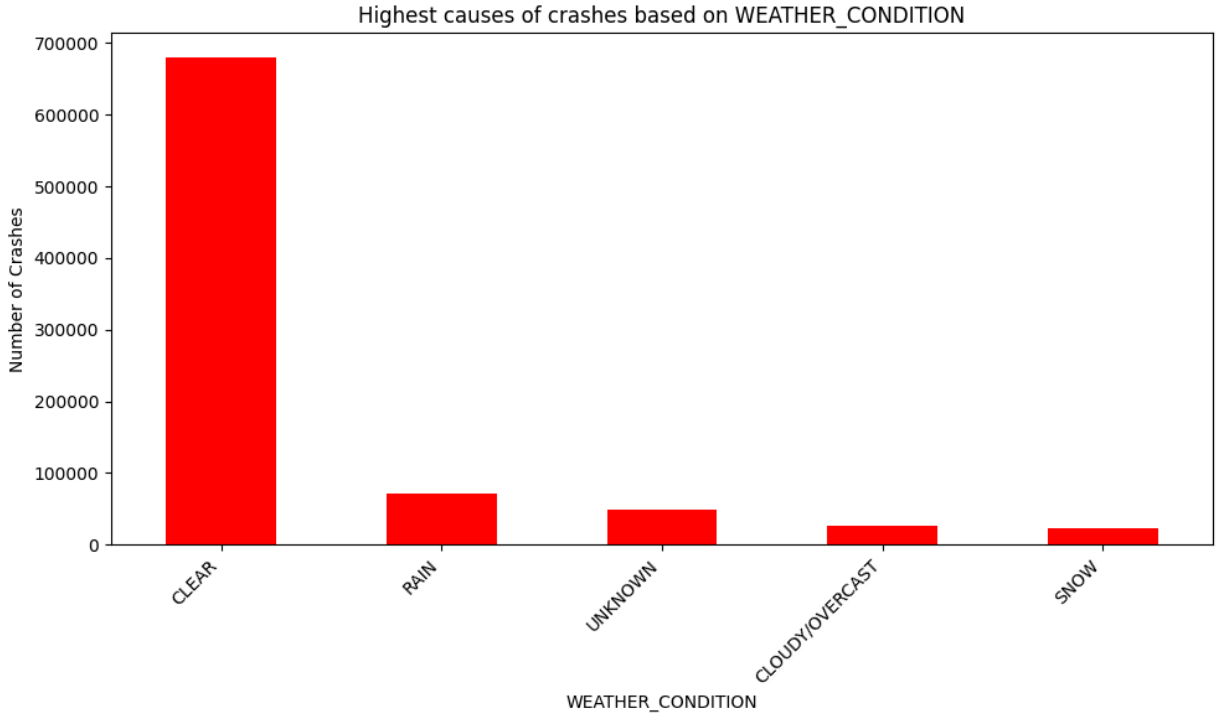

```
PHYSICAL_CONDITION is categorical with value counts:
PHYSICAL_CONDITION
NORMAL                 669400
UNKNOWN                169238
EMOTIONAL                5429
IMPAIRED - ALCOHOL       4218
REMOVED BY EMS           2771
Name: count, dtype: int64
```



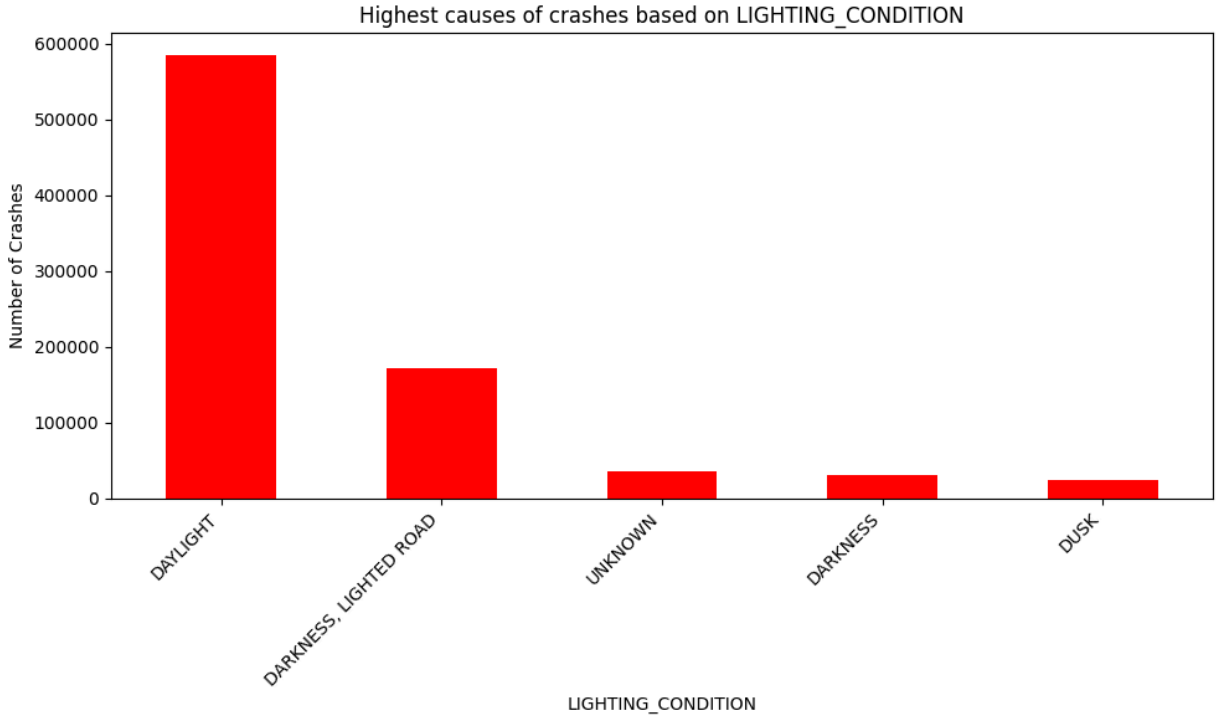

```
DRIVER_ACTION is categorical with value counts:
DRIVER_ACTION
NONE                    365242
UNKNOWN                 182539
OTHER                    88004
FAILED TO YIELD          73757
FOLLOWED TOO CLOSELY     41996
Name: count, dtype: int64
```



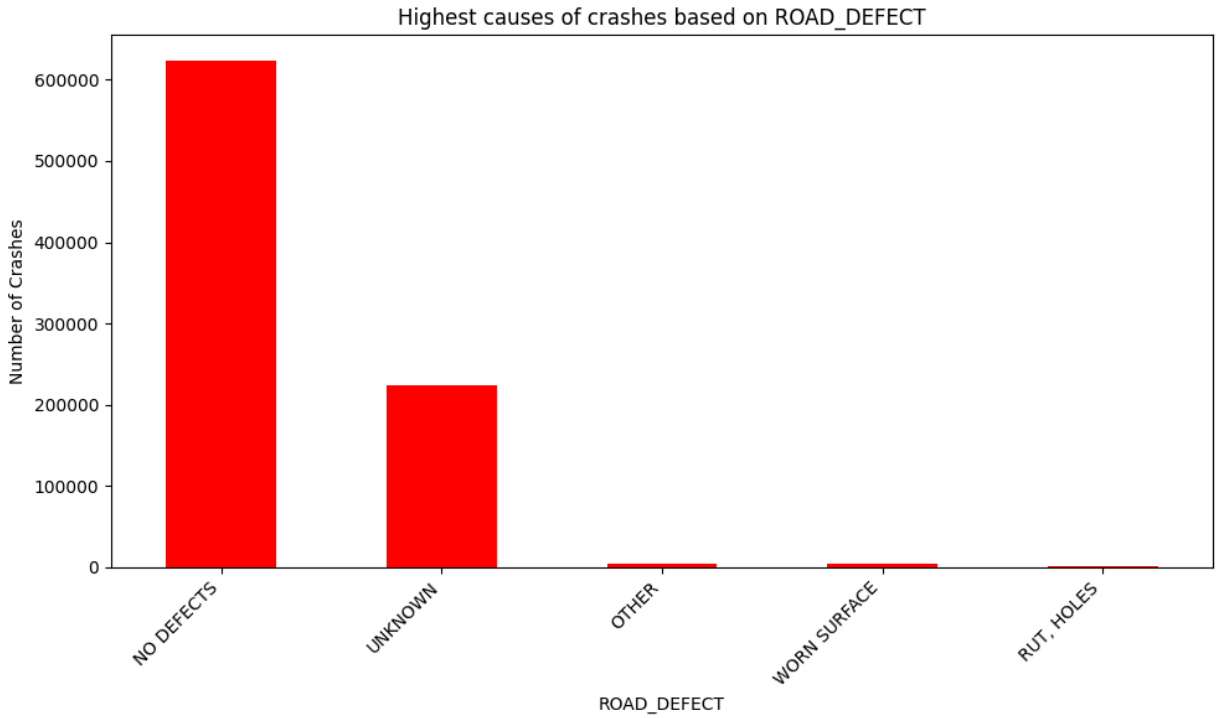

```
WEATHER_CONDITION is categorical with value counts:
WEATHER_CONDITION
CLEAR              679865
RAIN                71070
UNKNOWN             49129
CLOUDY/OVERCAST     26559
SNOW                23355
Name: count, dtype: int64
```



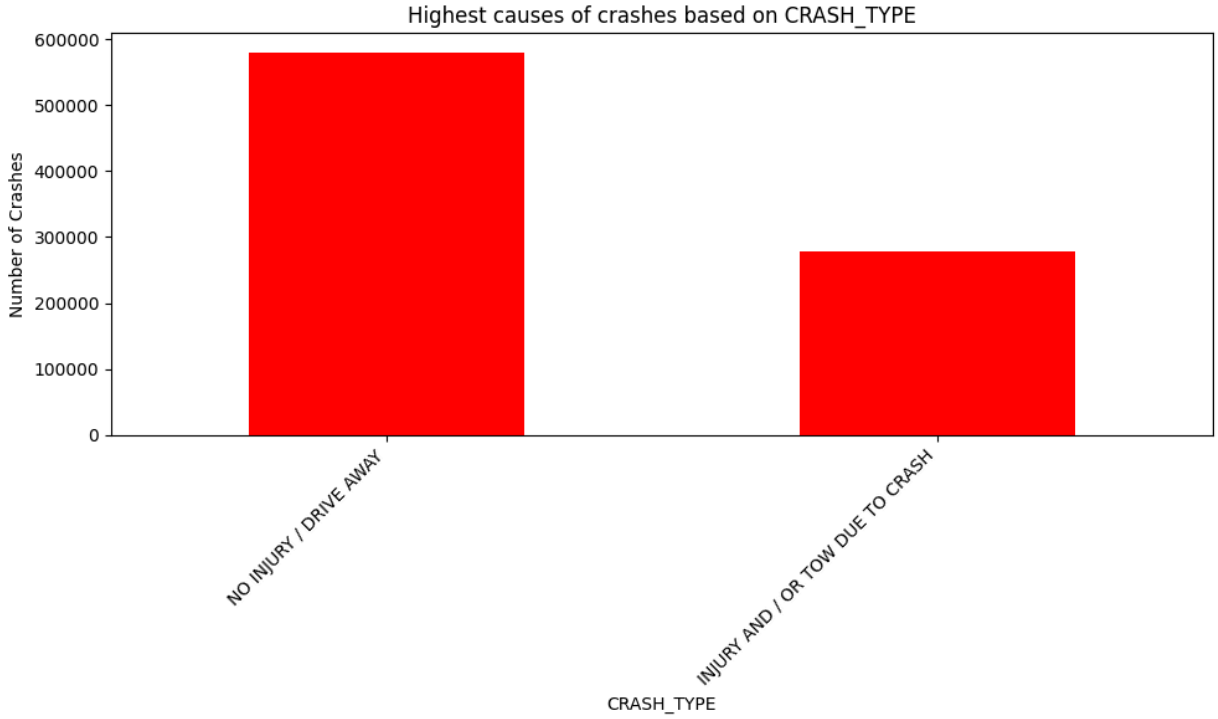

```
LIGHTING_CONDITION is categorical with value counts:
LIGHTING_CONDITION
DAYLIGHT                  584611
DARKNESS, LIGHTED ROAD    171213
UNKNOWN                    34822
DARKNESS                   30907
DUSK                       23364
Name: count, dtype: int64
```



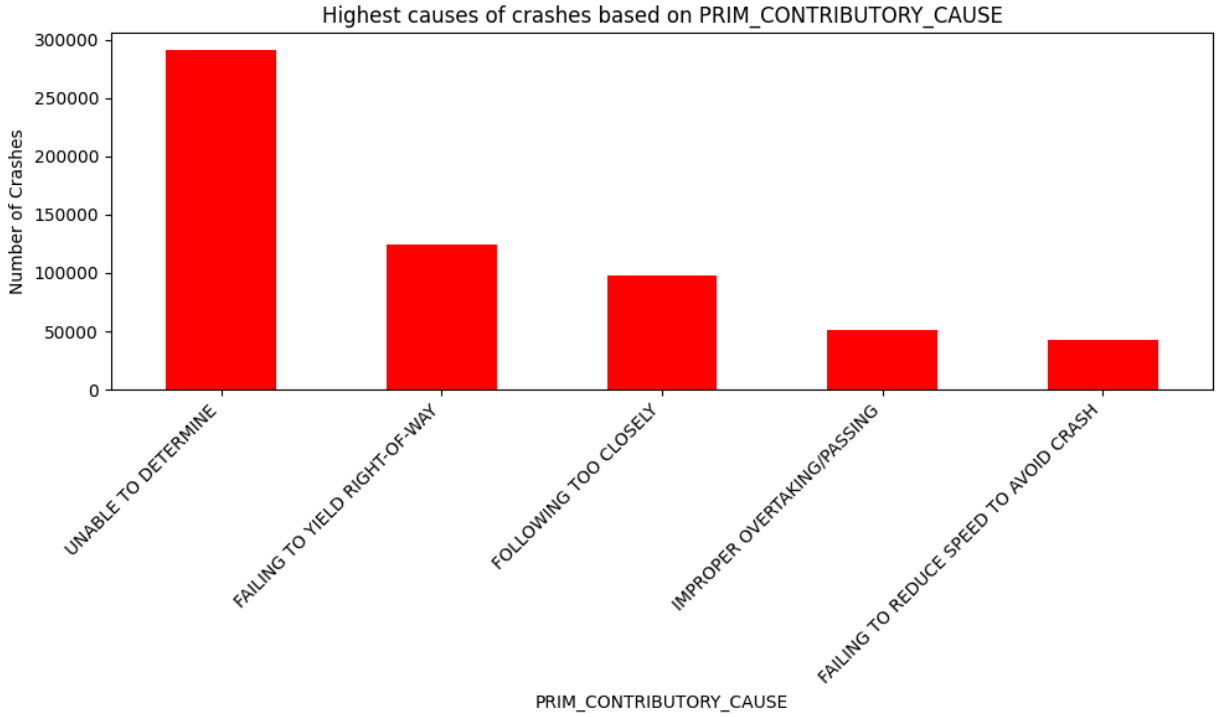

```
ROAD_DEFECT is categorical with value counts:
ROAD_DEFECT
NO DEFECTS      623923
UNKNOWN         224207
OTHER             3855
WORN SURFACE      3766
RUT, HOLES        1552
Name: count, dtype: int64
```



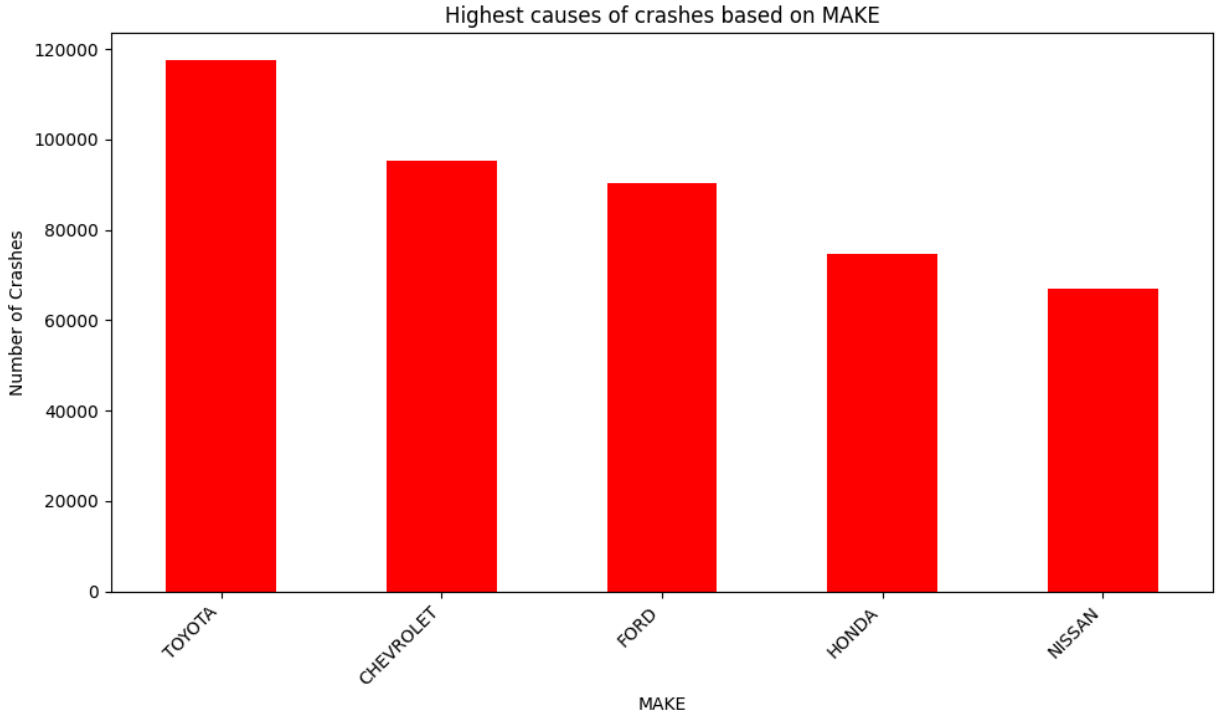

```
CRASH_TYPE is categorical with value counts:
CRASH_TYPE
NO INJURY / DRIVE AWAY              580012
INJURY AND / OR TOW DUE TO CRASH    278538
Name: count, dtype: int64
```



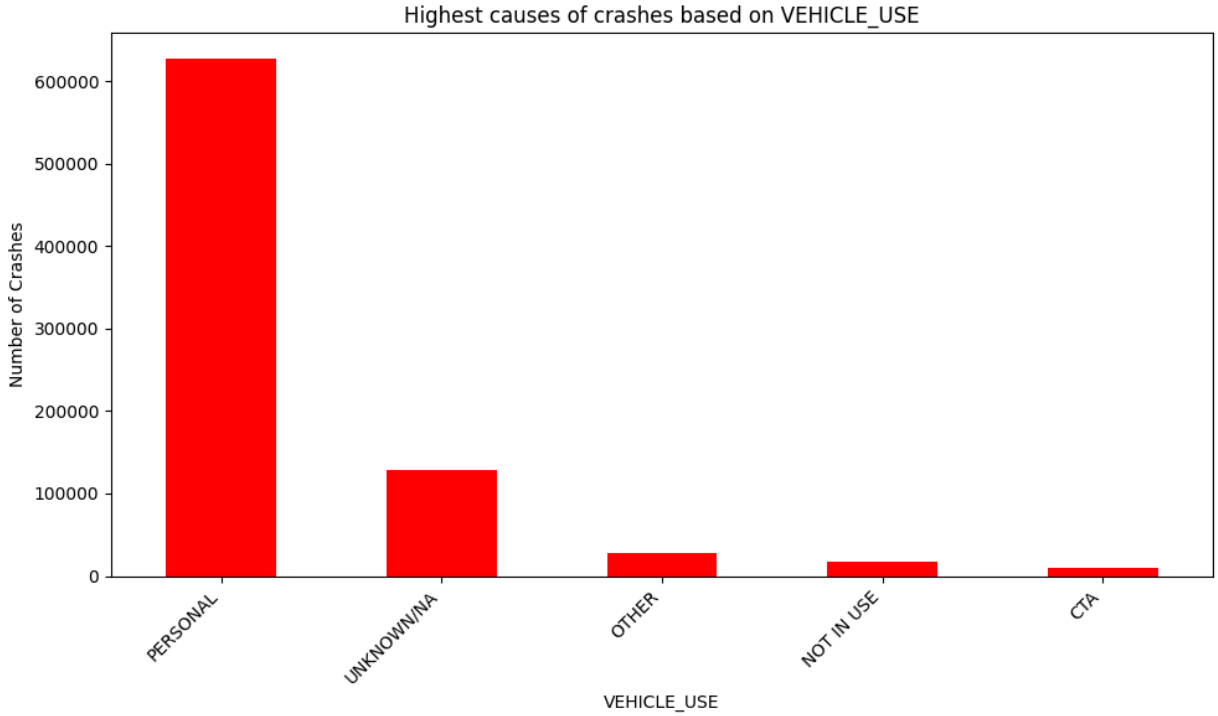

```
PRIM_CONTRIBUTORY_CAUSE is categorical with value counts:
PRIM_CONTRIBUTORY_CAUSE
UNABLE TO DETERMINE                       291352
FAILING TO YIELD RIGHT-OF-WAY             124226
FOLLOWING TOO CLOSELY                      98174
IMPROPER OVERTAKING/PASSING                50970
FAILING TO REDUCE SPEED TO AVOID CRASH     42900
Name: count, dtype: int64
```



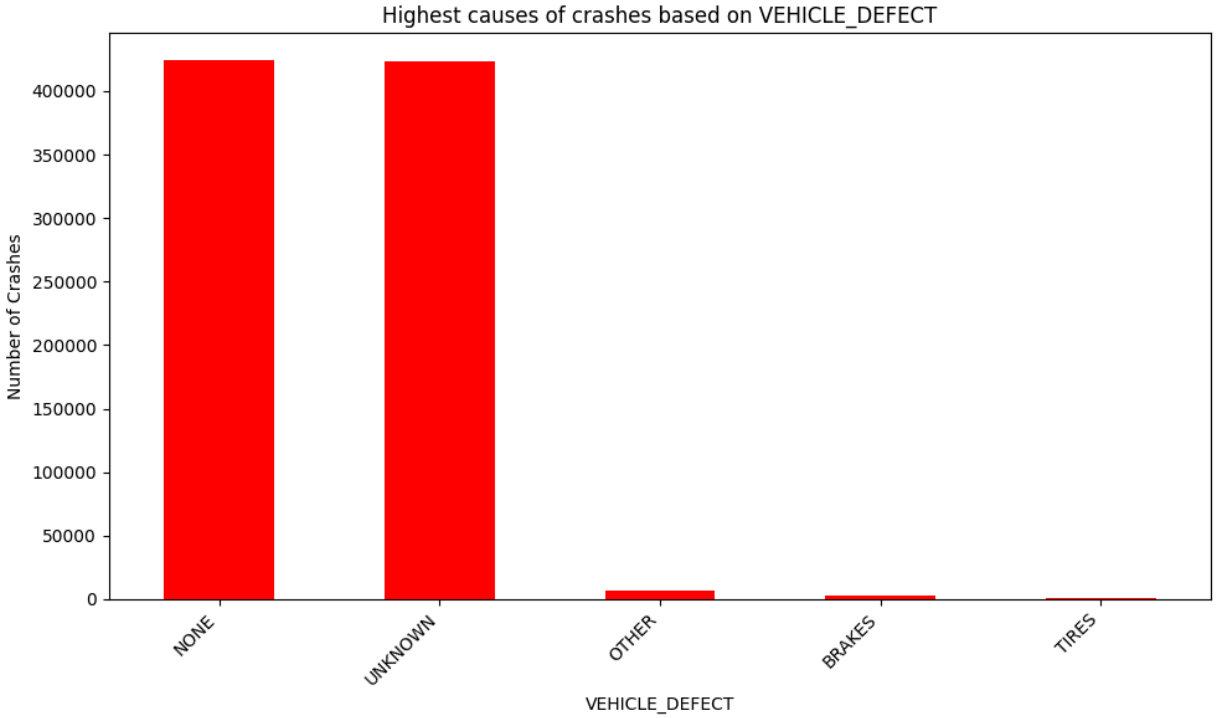

```
MAKE is categorical with value counts:
MAKE
TOYOTA       117635
CHEVROLET     95272
FORD          90249
HONDA         74679
NISSAN        66856
Name: count, dtype: int64
```




```
VEHICLE_USE is categorical with value counts:
VEHICLE_USE
PERSONAL      626883
UNKNOWN/NA    128992
OTHER          28364
NOT IN USE     17355
CTA             9736
Name: count, dtype: int64
```




```
VEHICLE_DEFECT is categorical with value counts:
VEHICLE_DEFECT
NONE       424366
UNKNOWN    423479
OTHER        6841
BRAKES       2508
TIRES         379
Name: count, dtype: int64
```



Summary Findings on the influence of categorical factors on occurence of accidents. Most accidents:

1. Occured when the weather was clear. However, when not clear, they occured in rainy conditions.
2. Occured in the daylight followed by darkly lighted roads.
3. Occured where roads had no defects followed by roads with worn out surfaces and potholes.
4. Did not involve any crash type injuries.
5. Did not have a primary cause but in those with a cause, the most common was either failing to give right of way or following too closely.
6. Involved Chevrolets, Fords and Nissans.
7. Involved personal cars.
8. Involved cars with no defects but of those which had brakes and tires had a defect.
9. Involved more men than women.
10. Were not caused by distractions, but of those were, the most common distractions were moving or parked vehicles.
11. Involved drivers with normal physical conditions, or drivers with impaired alcohol levels, emotionally unstable or fatigued.
12. Had no preceding driver action. Of those which had, drivers either failed to yield or followed too closely.

In [25]:
```
# Summarizing the impact of various numerical factors on accident occurrence.
# We will do this by checking the value counts of each numerical factor on accident
pd.cut(merged_df['AGE'], bins=10).value_counts()
```

Out[25]:
```
AGE
(22.8, 33.7]     259792
(33.7, 44.6]     203605
(44.6, 55.5]     147259
(55.5, 66.4]     104247
(11.9, 22.8]      82895
(66.4, 77.3]      47329
(77.3, 88.2]      11599
```

```
(88.2, 99.1]        977
(0.891, 11.9]       814
(99.1, 110.0]        33
Name: count, dtype: int64
```

In [26]:

```
# Illustrating the distribution of age in a histogram
plt.figure(figsize=(10, 6))
plt.hist(merged_df['AGE'], bins=10, color='blue', edgecolor='black')
plt.xlabel('Age')
plt.ylabel('Number of Crashes')
plt.title('Distribution of Age in Crashes')
plt.show()
```

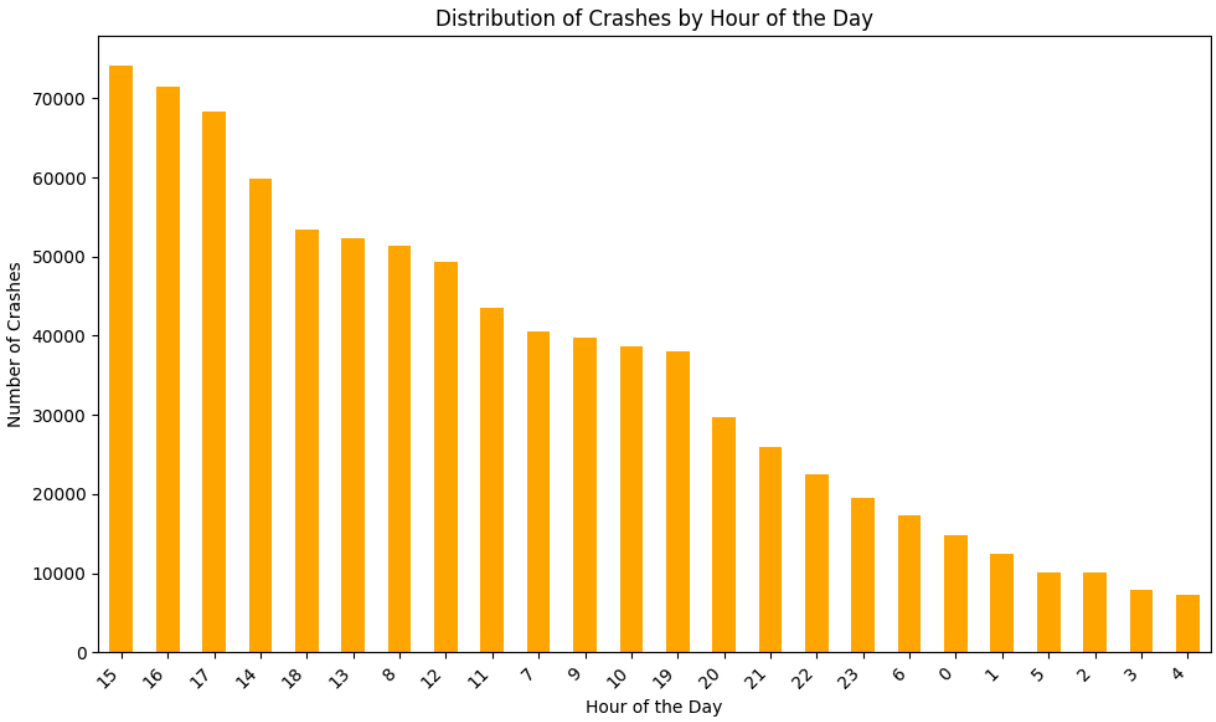
In [27]:

```
# We will now check the distribution of crashes by hour of the day to see if there a
merged_df['CRASH_HOUR'].value_counts()
```

Out[27]:

```
CRASH_HOUR
15    74142
16    71539
17    68384
14    59799
18    53414
13    52381
8     51447
12    49398
11    43523
7     40552
9     39707
10    38598
19    37980
20    29785
21    26000
22    22434
23    19569
6     17374
0     14840
1     12376
5     10119
2     10017
3      7844
4      7328
Name: count, dtype: int64
```

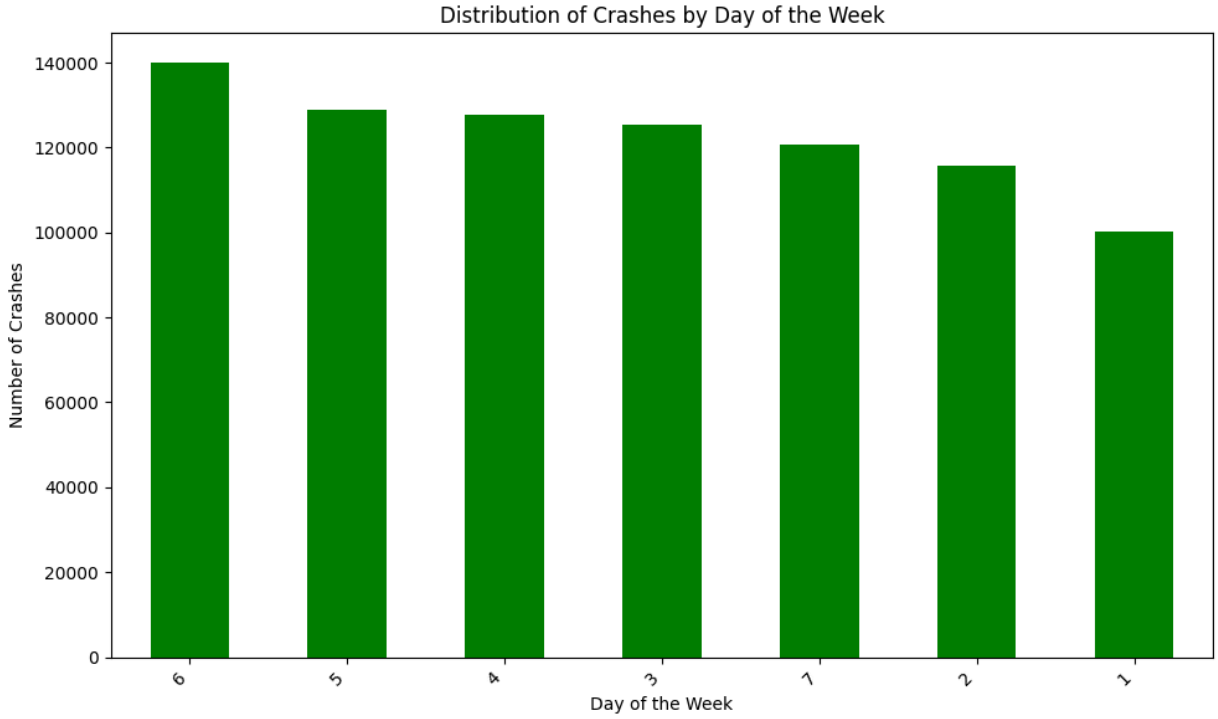
In [ ]:
```
#illustrating the distribution of crashes by hour of the day in a bar chart
plt.figure(figsize=(10, 6))
merged_df['CRASH_HOUR'].value_counts().plot(kind='bar', color='orange')
plt.xlabel('Hour of the Day')
plt.ylabel('Number of Crashes')
plt.title('Distribution of Crashes by Hour of the Day')
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()

# Each hour is represented in the 24 hour clock system
```



In [29]:
```
# We will now check the distribution of crashes by day of the week to see if there a
merged_df['CRASH_DAY_OF_WEEK'].value_counts()
```

Out[29]:
```
CRASH_DAY_OF_WEEK
6    139953
5    128919
4    127634
3    125343
7    120724
2    115638
1    100339
Name: count, dtype: int64
```

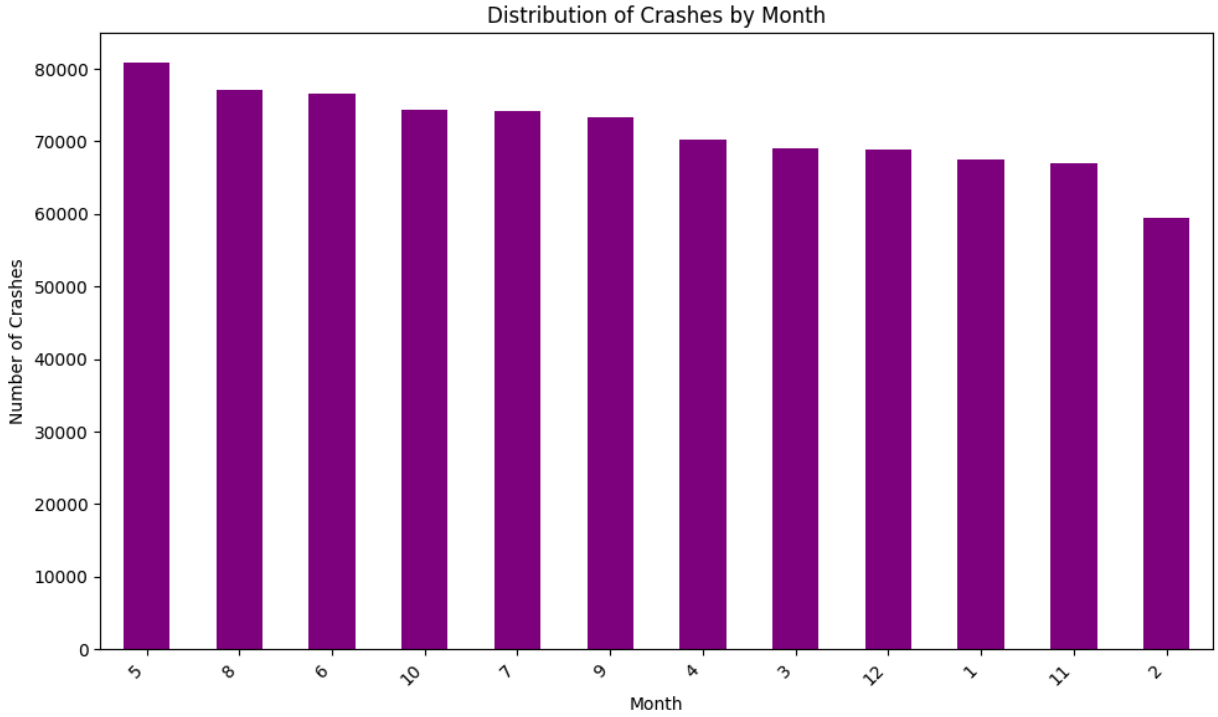
In [ ]:
```
# illustrating the distribution of crashes by day of the week in a bar chart
plt.figure(figsize=(10, 6))
merged_df['CRASH_DAY_OF_WEEK'].value_counts().plot(kind='bar', color='green')
plt.xlabel('Day of the Week')
plt.ylabel('Number of Crashes')
plt.title('Distribution of Crashes by Day of the Week')
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()

# For the days of the week: 1: Sunday, 2: Monday, 3: Tuesday, 4: Wednesday 5: Thursd
```



In [31]:
```
# We will now check the distribution of crashes by month to see if there are any pat
merged_df['CRASH_MONTH'].value_counts()
```

Out[31]:
```
CRASH_MONTH
5     80877
8     77073
6     76508
10    74430
7     74165
9     73328
4     70227
3     69060
12    68835
1     67530
11    67020
2     59497
Name: count, dtype: int64
```

In [ ]:
```
#illustrating the distribution of crashes by hour of the day in a bar chart
plt.figure(figsize=(10, 6))
merged_df['CRASH_MONTH'].value_counts().plot(kind='bar', color='purple')
plt.xlabel('Month')
plt.ylabel('Number of Crashes')
plt.title('Distribution of Crashes by Month')
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()

# For the months of the year, 1: January --> 12: December
```



Summary findings on the influence of numerical factors on the occurence of accidents.

Most accidents:

1. Involved drivers aged 20-40 years.
2. Occured between 3:00 p.m -5:00 p.m.
3. Occured on Fridays and Saturdays.
4. Occured in May and August.

5.7 Data Modeling

In this section, we will compare the following models to determine the most suitable for prediction:

1. Decision Tree - base model
2. Random forest classifier
3. Neural Networks

The selection of these algorithms stems from the fact that:

- Our target variable (Primary contributory cause) is a multi-class problem with 40 possible outcomes.

Prior to the training of each of the models, we will:

- Define our features and targets
- Split our training and test datasets
- Fit our models on the training dataset
- Predict on our x-test values
- Compare accuracy scores

Having done the above, we will then rerun the above steps with hyper parameter tuning, get the best model and retrain and predict on our data to get the best accuracy scores. These steps will be repeated for each model.

Challenges

We are working with standard laptops with approx. 16 GB worth of ram. Modelling with 858550 entries might prove difficult with current specifications.

- We will therefore randomly fraction and use 50% of the data.
- Random sampling will enable equal representation.

In [33]:
```
# Calculate our sample fraction

merged_df = merged_df.sample(frac=0.5, random_state=42)
```

In [34]:
```
# Re-checking our columns
merged_df.columns
```

Out[34]:
```
Index(['PERSON_TYPE', 'SEX', 'AGE', 'DRIVER_VISION', 'PHYSICAL_CONDITION',
       'DRIVER_ACTION', 'WEATHER_CONDITION', 'LIGHTING_CONDITION',
       'ROAD_DEFECT', 'CRASH_TYPE', 'PRIM_CONTRIBUTORY_CAUSE', 'CRASH_HOUR',
       'CRASH_DAY_OF_WEEK', 'CRASH_MONTH', 'OCCUPANT_CNT', 'MAKE',
       'VEHICLE_USE', 'VEHICLE_DEFECT'],
      dtype='object')
```

In [35]:
```
# Defining our features and targets
X = merged_df.drop('PRIM_CONTRIBUTORY_CAUSE', axis=1)

y = merged_df['PRIM_CONTRIBUTORY_CAUSE']
```

In [36]:
```
# Split the data into train-test split
# We will use the common 70-30 train- test split ratio which is common in machine le

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_stat
```

In [37]:
```
# Our X data contains both categorical and numerical data types. We will therefore s
# We will one hot encode the categorical variables because most machine learning mod
# We will StandardScale the numerical variables because the numerical data are all i

# Initialise StandardScaler and OneHot Encoder
scaler = StandardScaler()
encoder = OneHotEncoder(sparse_output=False, handle_unknown='ignore', drop='first')

# Seperate numerical from categorical columns based on data type
num_cols = X.select_dtypes(include=['int64', 'float64']).columns
cat_cols = X.select_dtypes(include=['object']).columns

# Specify and scale numerical features
X_train[num_cols] = scaler.fit_transform(X_train[num_cols])
X_test[num_cols] = scaler.transform(X_test[num_cols])  # only transform to prevent d

# One-hot encode categorical features
X_train_encoded = pd.DataFrame(encoder.fit_transform(X_train[cat_cols]),
                               columns=encoder.get_feature_names_out(cat_cols),
                               index=X_train.index)
X_test_encoded = pd.DataFrame(encoder.transform(X_test[cat_cols]),
                              columns=encoder.get_feature_names_out(cat_cols),
                              index=X_test.index)
```

```
# Combine final numerical and encoded categorical features in X_train and X_test
X_train_final = pd.concat([X_train[num_cols], X_train_encoded], axis=1)
X_test_final = pd.concat([X_test[num_cols], X_test_encoded], axis=1)
```

```
/Users/administrator/Library/Python/3.9/lib/python/site-packages/sklearn/preprocessin
g/_encoders.py:246: UserWarning: Found unknown categories in columns [9] during trans
form. These unknown categories will be encoded as all zeros
  warnings.warn(
```

5.7.1. Decision Trees

Starting off with the base model - decision trees

- Decision trees can directly predict multiple classes without needing to transform the problem.
- Decision trees can model complex, non-linear relationships between features and classes.
- Trees can focus on splitting to maximize information gain for each class, making them robust.
- One can tune several hyper parameters such as tree depth, min samples per leaf, etc., to control complexity and reduce overfitting in multiclass problems.

In [38]:

```
# Initialising base decision tree
base_DT = DecisionTreeClassifier(random_state=42)

# Fitting on the prepared training data
base_DT.fit(X_train_final, y_train)

# Predicting on training and testing data
y_train_base_DT_pred = base_DT.predict(X_train_final)
y_test_base_DT_pred = base_DT.predict(X_test_final)

# Assessing accuracy of base decision tree algorithm on training and testing data
print("Base Tree Training Accuracy:", accuracy_score(y_train, y_train_base_DT_pred))
print("Base Tree Test Accuracy:", accuracy_score(y_test, y_test_base_DT_pred))
```

```
Base Tree Training Accuracy: 0.9993111297472146
Base Tree Test Accuracy: 0.40016927700084637
```

The result from our base decision tree indicates underfitting since the training accuracy was high at 99% but the test accuracy is low at 40%. We will attempt to address this by using hyper parameter tuning by giving higher values of Max Depth (in an attempt to learn more complex data) and lower values of min samples split and min sample leaf.

In [39]:

```
# Defining hyperparameter grid of our base decision tree
param_grid = {
    'max_depth': [None, 15],
    'min_samples_split': [2, 5],
    'min_samples_leaf': [1, 2],
    'criterion': ['gini', 'entropy'],
}

# Initialize GridSearchCV
grid_tree_DT = GridSearchCV(
    estimator=DecisionTreeClassifier(random_state=42),
    param_grid=param_grid,
    cv=2,
    scoring='accuracy',
    n_jobs=-1
)
```

```
# Fitting our GridSearchCV on training data
grid_tree_DT.fit(X_train_final, y_train)

# Best parameters
print("Best Hyperparameters:", grid_tree_DT.best_params_)

# Best estimator predictions
best_tree = grid_tree_DT.best_estimator_
y_train_pred_best = best_tree.predict(X_train_final)
y_test_pred_best = best_tree.predict(X_test_final)

# Evaluation
train_dt_accuracy=accuracy_score(y_train, y_train_pred_best)
test_dt_accuracy=accuracy_score(y_test, y_test_pred_best)
print("Tuned Tree Training Accuracy:", train_dt_accuracy)
print("Tuned Tree Test Accuracy:", test_dt_accuracy)
```

```
Best Hyperparameters: {'criterion': 'gini', 'max_depth': 15, 'min_samples_leaf': 1,
'min_samples_split': 2}
Tuned Tree Training Accuracy: 0.4808979939565779
Tuned Tree Test Accuracy: 0.46378015731890077
```

Despite hyper parameter tuning, the decision tree only improves on testing accuracy from 36.3% to 45.8%. On the other hand, the test accuracy for this model is still low.

- In this case, we will consider using a random forest classifier because:

Random Forest aggregates many trees, each capturing different patterns, making the model more flexible and better at fitting the data as compared to a single shallow decision tree that may underfit.

5.7.2 Random Forest Classifier

A Random Forest classifier is an ensemble learning method that builds many decision trees and combines their outputs.

- By averaging the multiple trees, it reduces errors and overfitting from individual models.
- It also handles large and omplex data better.
- It is robust to noise and outliers.
- Captures broader patterns that apply better to unseen data.

We shall start off with the base model of the Random Forest Classifier.

In [40]:

```
# Create the Random Forest classifier with default parameters
base_rf = RandomForestClassifier(random_state=42)

# Fit the model on the training data
base_rf.fit(X_train_final, y_train)

# Predict on training and testing data
y_train_base_rf_pred = base_rf.predict(X_train_final)
y_test_base_rf_pred = base_rf.predict(X_test_final)

# Calculate training and testing accuracy of base_rf classifier
base_rf_train_accuracy = accuracy_score(y_train, y_train_base_rf_pred)
base_rf_test_accuracy = accuracy_score(y_test, y_test_base_rf_pred)

print("Training Accuracy:", base_rf_train_accuracy)
print("Testing Accuracy:", base_rf_test_accuracy)
```

```
Training Accuracy: 0.9993111297472146
Testing Accuracy: 0.49159438745797196
```

The base random forest classifier, similar to the base decision tree seems to be underfitting on the data with a training accuracy of 99.9% but a testing accuracy of 49.1%. We will therefore need to tune the model with hyper parameter tuning.

In [41]:

```
# Running a random forest classifier with hyper parameter tuning
param_grid = {
    'n_estimators': [50],
    'max_depth': [10, None],
    'min_samples_split': [2, 5],
    'min_samples_leaf': [1, 2]
}

# Instantiating the GridSearchCV
grid_rf = GridSearchCV(
    estimator=base_rf,
    param_grid=param_grid,
    cv=2,
    n_jobs=-1,
    scoring='accuracy'
)

# Fit GridSearchCV
grid_rf.fit(X_train_final, y_train)

# Best model after tuning
best_rf = grid_rf.best_estimator_

# Predictions
y_train_rf_pred = best_rf.predict(X_train_final)
y_test_rf_pred = best_rf.predict(X_test_final)

# Accuracy
train_rf_accuracy = accuracy_score(y_train, y_train_rf_pred)
test_rf_accuracy = accuracy_score(y_test, y_test_rf_pred)

print("Best parameters:", grid_rf.best_params_)
print("Training Accuracy:", train_rf_accuracy)
print("Testing Accuracy:", test_rf_accuracy)
```

```
Best parameters: {'max_depth': None, 'min_samples_leaf': 1, 'min_samples_split': 5,
'n_estimators': 50}
Training Accuracy: 0.9012852255634093
Testing Accuracy: 0.4869043274345216
```

The tuned random forest classifier, similar to the base random forest classifier seems to be underfitting on the data with a training accuracy of 90.0% with an unchanged testing accuracy of 48.7%. There are several reasons for the underperformance of both the decision tree and random forest classifier with the main one being potential class imbalance in the dataset as shown:

In [42]:

```
merged_df['PRIM_CONTRIBUTORY_CAUSE'].value_counts()  # As seen below, some classes h

# We will address this using neural networks where we will assign higher penalties t
```

Out[42]:

```
PRIM_CONTRIBUTORY_CAUSE
UNABLE TO DETERMINE                                                    1
45754
FAILING TO YIELD RIGHT-OF-WAY
61980
FOLLOWING TOO CLOSELY
```

```
49217
IMPROPER OVERTAKING/PASSING
25572
FAILING TO REDUCE SPEED TO AVOID CRASH
21465
IMPROPER TURNING/NO SIGNAL
18093
NOT APPLICABLE
17070
DRIVING SKILLS/KNOWLEDGE/EXPERIENCE
15176
IMPROPER LANE USAGE
15088
DISREGARDING TRAFFIC SIGNALS
11855
IMPROPER BACKING
11345
WEATHER
6052
DISREGARDING STOP SIGN
4560
OPERATING VEHICLE IN ERRATIC, RECKLESS, CARELESS, NEGLIGENT OR AGGRESSIVE MANNER
3667
DISTRACTION - FROM INSIDE VEHICLE
3246
DRIVING ON WRONG SIDE/WRONG WAY
2698
EQUIPMENT - VEHICLE CONDITION
2641
VISION OBSCURED (SIGNS, TREE LIMBS, BUILDINGS, ETC.)
2531
PHYSICAL CONDITION OF DRIVER
2498
UNDER THE INFLUENCE OF ALCOHOL/DRUGS (USE WHEN ARREST IS EFFECTED)
2151
DISTRACTION - FROM OUTSIDE VEHICLE
1436
DISREGARDING OTHER TRAFFIC SIGNS
899
CELL PHONE USE OTHER THAN TEXTING
594
EVASIVE ACTION DUE TO ANIMAL, OBJECT, NONMOTORIST
564
DISREGARDING ROAD MARKINGS
516
ROAD CONSTRUCTION/MAINTENANCE
448
TURNING RIGHT ON RED
413
RELATED TO BUS STOP
364
ROAD ENGINEERING/SURFACE/MARKING DEFECTS
324
DISTRACTION - OTHER ELECTRONIC DEVICE (NAVIGATION DEVICE, DVD PLAYER, ETC.)
296
HAD BEEN DRINKING (USE WHEN ARREST IS NOT MADE)
241
ANIMAL
229
TEXTING
125
OBSTRUCTED CROSSWALKS
58
DISREGARDING YIELD SIGN
52
PASSING STOPPED SCHOOL BUS
43
BICYCLE ADVANCING LEGALLY ON RED LIGHT
8
```

```
MOTORCYCLE ADVANCING LEGALLY ON RED LIGHT
6
Name: count, dtype: int64
```

5.7.3 Neural Networks

Neural networks are a type of machine learning model made of interconnected layers of nodes that learn patterns from data to make decisions.

When compared to the Random Forest Classifier, they:

- Are better for complex patterns.
- Scale better with large data.
- Provide more tools for overfitting such as regularization, dropout and early stopping.
- Handles underfitting with more flexibility by adding more layers (depth) and neurons (width).

Architecture Design

1. What is the task? It's a multiclass because we are predicting 40 possible categorical outcomes.
2. What is the output activation? Softmax
3. What is the loss function? Categorical crossentropy

In [43]:

```
# Given that we will be using categorical crossentropy as the loss function, we will

y_train_cat = pd.get_dummies(y_train)
y_test_cat  = pd.get_dummies(y_test)

y_test_cat = y_test_cat.reindex(columns=y_train_cat.columns, fill_value=0)

y_train_cat = y_train_cat.astype('int32')
y_test_cat  = y_test_cat.astype('int32')
```

In [44]:

```
# Build the baseline model (Dense)
model= keras.Sequential([
    #inputs
    keras.Input(shape=(X_train_final.shape[1],), name= "inputs"),
    #hidden layer 1
    layers.Dense(64, name= "Dense_1"),
    layers.BatchNormalization(name="Batch_1"),
    layers.Activation('relu', name = "relu_1"),
    layers.Dropout(0.2),
    #hidden layer 2
    layers.Dense(32, name= "Dense_2"),
    layers.BatchNormalization(name="Batch_2"),
    layers.Activation('relu', name = "relu_2"),
    layers.Dropout(0.2),
    #output layer
    layers.Dense(38, name= "Output", activation= 'softmax')
], name= "Numbers_Classification")

#Compile the model
model.compile(loss='categorical_crossentropy',
            metrics=['accuracy'],
            optimizer=keras.optimizers.Adam()
            )

#Fit our data
```

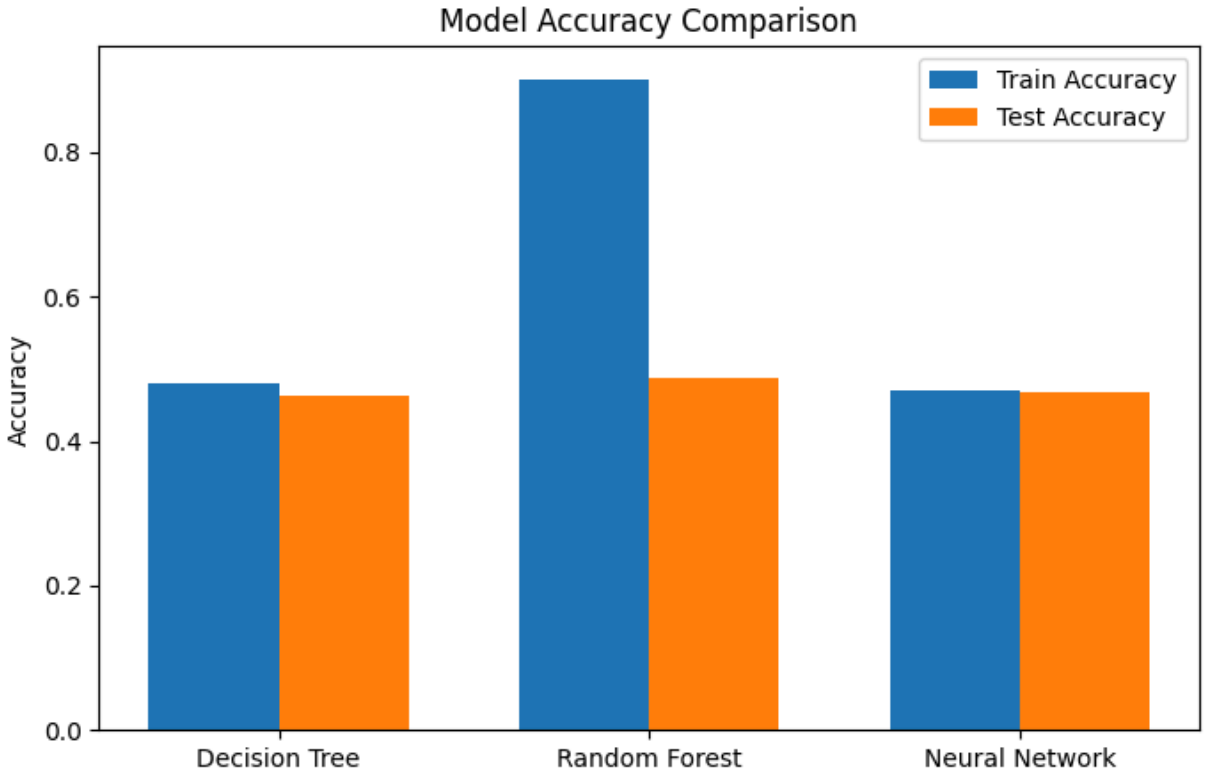
```
model.fit(X_train_final, y_train_cat, epochs=10, batch_size=100, validation_split=0.

# Evaluate on the test set
test_loss, test_acc = model.evaluate(X_test_final, y_test_cat)
train_loss, train_acc = model.evaluate(X_train_final, y_train_cat)
print("Test accuracy:", test_acc)
print("Train accuracy:", train_acc)
```

```
Epoch 1/10
2555/2555 ━━━━━━━━━━━━━━━━━━━━ 5s 2ms/step - accuracy: 0.3965 - loss: 2.2619 - val_ac
curacy: 0.4643 - val_loss: 1.9022
Epoch 2/10
2555/2555 ━━━━━━━━━━━━━━━━━━━━ 4s 1ms/step - accuracy: 0.4605 - loss: 1.9352 - val_ac
curacy: 0.4644 - val_loss: 1.8889
Epoch 3/10
2555/2555 ━━━━━━━━━━━━━━━━━━━━ 3s 1ms/step - accuracy: 0.4630 - loss: 1.9172 - val_ac
curacy: 0.4645 - val_loss: 1.8837
Epoch 4/10
2555/2555 ━━━━━━━━━━━━━━━━━━━━ 3s 1ms/step - accuracy: 0.4635 - loss: 1.9089 - val_ac
curacy: 0.4651 - val_loss: 1.8817
Epoch 5/10
2555/2555 ━━━━━━━━━━━━━━━━━━━━ 3s 1ms/step - accuracy: 0.4627 - loss: 1.9029 - val_ac
curacy: 0.4663 - val_loss: 1.8774
Epoch 6/10
2555/2555 ━━━━━━━━━━━━━━━━━━━━ 3s 1ms/step - accuracy: 0.4621 - loss: 1.9063 - val_ac
curacy: 0.4646 - val_loss: 1.8759
Epoch 7/10
2555/2555 ━━━━━━━━━━━━━━━━━━━━ 3s 1ms/step - accuracy: 0.4632 - loss: 1.8970 - val_ac
curacy: 0.4638 - val_loss: 1.8754
Epoch 8/10
2555/2555 ━━━━━━━━━━━━━━━━━━━━ 4s 1ms/step - accuracy: 0.4642 - loss: 1.8961 - val_ac
curacy: 0.4641 - val_loss: 1.8746
Epoch 9/10
2555/2555 ━━━━━━━━━━━━━━━━━━━━ 4s 1ms/step - accuracy: 0.4656 - loss: 1.8897 - val_ac
curacy: 0.4651 - val_loss: 1.8736
Epoch 10/10
2555/2555 ━━━━━━━━━━━━━━━━━━━━ 3s 1ms/step - accuracy: 0.4630 - loss: 1.8950 - val_ac
curacy: 0.4658 - val_loss: 1.8724
4025/4025 ━━━━━━━━━━━━━━━━━━━━ 2s 591us/step - accuracy: 0.4685 - loss: 1.8528
9391/9391 ━━━━━━━━━━━━━━━━━━━━ 5s 546us/step - accuracy: 0.4716 - loss: 1.8398
Test accuracy: 0.4686332941055298
Train accuracy: 0.4704684317111969
```

Results from our neural networks still indicate an underfitting picture.

In [45]:

```
# Illustrating training accuracy score of our three models
models = ['Decision Tree', 'Random Forest', 'Neural Network']

train_acc = [train_dt_accuracy, train_rf_accuracy, train_acc]
test_acc  = [test_dt_accuracy,  test_rf_accuracy,  test_acc]

x = np.arange(len(models))
width = 0.35

plt.figure(figsize=(8,5))

plt.bar(x - width/2, train_acc, width, label='Train Accuracy')
plt.bar(x + width/2, test_acc,  width, label='Test Accuracy')

plt.xticks(x, models)
plt.ylabel('Accuracy')
plt.title('Model Accuracy Comparison')
plt.legend()

plt.show()
```



All our models seem to be under-performing. The potential reasons could be:

1. Small sample size
2. Imbalanced classes where one class dominates

5.7.4 Running the algorithms with higher sample size - increased to 80% of the data from the 50% of the data used previously

In [46]:
```
# Recall that our model uses 80% of the data. We will now attempt with 80% of our da
merged_df_2 = merged_df.sample(frac=0.8, random_state=42)
```

In [47]:
```
# Defining our features and targets
X = merged_df_2.drop('PRIM_CONTRIBUTORY_CAUSE', axis=1)

y = merged_df_2['PRIM_CONTRIBUTORY_CAUSE']
```

In [48]:
```
# Split the data into train-test split
# We will use the common 70-30 train- test split ratio which is common in machine le

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_stat
```

In [49]:
```
# Our X data contains both categorical and numerical data types. We will therefore s
# We will one hot encode the categorical variables because most machine learning mod
# We will StandardScale the numerical variables because the numerical data are all i

# Initialise StandardScaler and OneHot Encoder
scaler = StandardScaler()
encoder = OneHotEncoder(sparse_output=False, handle_unknown='ignore', drop='first')

# Seperate numerical from categorical columns based on data type
num_cols = X.select_dtypes(include=['int64', 'float64']).columns
cat_cols = X.select_dtypes(include=['object']).columns

# Specify and scale numerical features
```

```
X_train[num_cols] = scaler.fit_transform(X_train[num_cols])
X_test[num_cols] = scaler.transform(X_test[num_cols])  # only transform to prevent d

# One-hot encode categorical features
X_train_encoded_2 = pd.DataFrame(encoder.fit_transform(X_train[cat_cols]),
                                 columns=encoder.get_feature_names_out(cat_cols),
                                 index=X_train.index)
X_test_encoded_2 = pd.DataFrame(encoder.transform(X_test[cat_cols]),
                                columns=encoder.get_feature_names_out(cat_cols),
                                index=X_test.index)

# Combine final numerical and encoded categorical features in X_train and X_test
X_train_final_2 = pd.concat([X_train[num_cols], X_train_encoded_2], axis=1)
X_test_final_2 = pd.concat([X_test[num_cols], X_test_encoded_2], axis=1)
```

```
/Users/administrator/Library/Python/3.9/lib/python/site-packages/sklearn/preprocessin
g/_encoders.py:246: UserWarning: Found unknown categories in columns [9] during trans
form. These unknown categories will be encoded as all zeros
  warnings.warn(
```

Decision Trees

In [50]:

```
# Initialising base decision tree
base_DT_2 = DecisionTreeClassifier(random_state=42)

# Fitting on the prepared training data
base_DT_2.fit(X_train_final_2, y_train)

# Predicting on training and testing data
y_train_base_DT_pred_2 = base_DT_2.predict(X_train_final_2)
y_test_base_DT_pred_2 = base_DT_2.predict(X_test_final_2)

# Assessing accuracy of base decision tree algorithm on training and testing data
print("Base Tree Training Accuracy:", accuracy_score(y_train, y_train_base_DT_pred_2
print("Base Tree Test Accuracy:", accuracy_score(y_test, y_test_base_DT_pred_2))
```

```
Base Tree Training Accuracy: 0.9994134628984085
Base Tree Test Accuracy: 0.38521344126725293
```

In [51]:

```
# Defining hyperparameter grid of our base decision tree
param_grid = {
    'max_depth': [None, 15],
    'min_samples_split': [2, 5],
    'min_samples_leaf': [1, 2],
    'criterion': ['gini', 'entropy'],
}

# Initialize GridSearchCV
grid_tree_DT_2 = GridSearchCV(
    estimator=DecisionTreeClassifier(random_state=42),
    param_grid=param_grid,
    cv=2,
    scoring='accuracy',
    n_jobs=-1
)

# Fitting our GridSearchCV on training data
grid_tree_DT_2.fit(X_train_final_2, y_train)

# Best parameters
print("Best Hyperparameters:", grid_tree_DT_2.best_params_)

# Best estimator predictions
best_tree_2 = grid_tree_DT_2.best_estimator_
y_train_pred_best_2 = best_tree_2.predict(X_train_final_2)
```

```
y_test_pred_best_2 = best_tree_2.predict(X_test_final_2)

# Evaluation
train_dt_accuracy_2=accuracy_score(y_train, y_train_pred_best_2)
test_dt_accuracy_2=accuracy_score(y_test, y_test_pred_best_2)
print("Tuned Tree Training Accuracy:", train_dt_accuracy_2)
print("Tuned Tree Test Accuracy:", test_dt_accuracy_2)
```

```
Best Hyperparameters: {'criterion': 'gini', 'max_depth': 15, 'min_samples_leaf': 1,
'min_samples_split': 2}
Tuned Tree Training Accuracy: 0.4830486617802441
Tuned Tree Test Accuracy: 0.4576708791955429
```

Random Forest Classifier

In [52]:

```
# Create the Random Forest classifier with default parameters
base_rf_2 = RandomForestClassifier(random_state=42)

# Fit the model on the training data
base_rf_2.fit(X_train_final_2, y_train)

# Predict on training and testing data
y_train_base_rf_pred_2 = base_rf_2.predict(X_train_final_2)
y_test_base_rf_pred_2 = base_rf_2.predict(X_test_final_2)

# Calculate training and testing accuracy of base_rf classifier
base_rf_train_accuracy_2 = accuracy_score(y_train, y_train_base_rf_pred_2)
base_rf_test_accuracy_2 = accuracy_score(y_test, y_test_base_rf_pred_2)

print("Training Accuracy:", base_rf_train_accuracy_2)
print("Testing Accuracy:", base_rf_test_accuracy_2)
```

```
Training Accuracy: 0.9994134628984085
Testing Accuracy: 0.48161629103333137
```

In [53]:

```
# Running a random forest classifier with hyper parameter tuning
param_grid = {
    'n_estimators': [50],
    'max_depth': [10, None],
    'min_samples_split': [2, 5],
    'min_samples_leaf': [1, 2]
}

# Instantiating the GridSearchCV
grid_rf_2 = GridSearchCV(
    estimator=base_rf,
    param_grid=param_grid,
    cv=2,
    n_jobs=-1,
    scoring='accuracy'
)

# Fit GridSearchCV
grid_rf_2.fit(X_train_final_2, y_train)

# Best model after tuning
best_rf_2 = grid_rf_2.best_estimator_

# Predictions
y_train_rf_pred_2 = best_rf_2.predict(X_train_final_2)
y_test_rf_pred_2= best_rf_2.predict(X_test_final_2)

# Accuracy
train_rf_accuracy_2 = accuracy_score(y_train, y_train_rf_pred_2)
test_rf_accuracy_2 = accuracy_score(y_test, y_test_rf_pred_2)
```

```
print("Best parameters:", grid_rf_2.best_params_)
print("Training Accuracy:", train_rf_accuracy_2)
print("Testing Accuracy:", test_rf_accuracy_2)
```

```
Best parameters: {'max_depth': None, 'min_samples_leaf': 1, 'min_samples_split': 5,
'n_estimators': 50}
Training Accuracy: 0.8993527292694493
Testing Accuracy: 0.4793741385669637
```

Neural Network

In [ ]:
```
# Given that we will be using categorical crossentropy as the loss function, we will

y_train_cat_2 = pd.get_dummies(y_train)
y_test_cat_2  = pd.get_dummies(y_test)

y_test_cat_2 = y_test_cat_2.reindex(columns=y_train_cat_2.columns, fill_value=0)

y_train_cat_2 = y_train_cat_2.astype('int32')
y_test_cat_2  = y_test_cat_2.astype('int32')
```

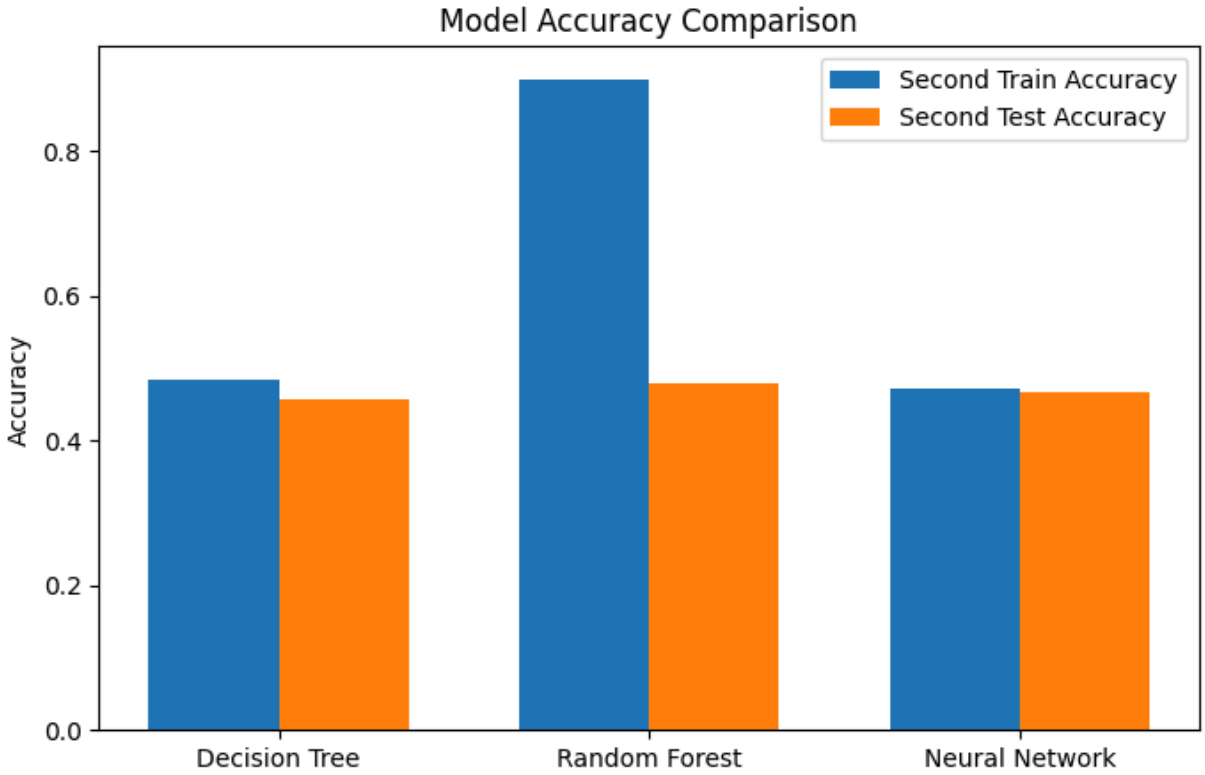
In [ ]:
```
# Build the baseline model (Dense) this time with 80% of the data.
model= keras.Sequential([
    #inputs
    keras.Input(shape=(X_train_final_2.shape[1],), name= "inputs"),
    #hidden layer 1
    layers.Dense(64, name= "Dense_1"),
    layers.BatchNormalization(name="Batch_1"),
    layers.Activation('relu', name = "relu_1"),
    layers.Dropout(0.2),
    #hidden layer 2
    layers.Dense(32, name= "Dense_2"),
    layers.BatchNormalization(name="Batch_2"),
    layers.Activation('relu', name = "relu_2"),
    layers.Dropout(0.2),
    #output layer
    layers.Dense(38, name= "Output", activation= 'softmax')
], name= "Numbers_Classification")

#Compile the model
model.compile(loss='categorical_crossentropy',
            metrics=['accuracy'],
            optimizer=keras.optimizers.Adam()
            )

#Fit our data
model.fit(X_train_final_2, y_train_cat_2, epochs=10, batch_size=100, validation_spli

# Evaluate on the test set
test_loss_2, test_acc_2 = model.evaluate(X_test_final_2, y_test_cat_2)
train_loss_2, train_acc_2 = model.evaluate(X_train_final_2, y_train_cat_2)
print("Test accuracy:", test_acc_2)
print("Train accuracy:", train_acc_2)
```

```
Epoch 1/10
2044/2044 ━━━━━━━━━━━━━━━━━━━━ 5s 2ms/step - accuracy: 0.3978 - loss: 2.2793 - val_ac
curacy: 0.4622 - val_loss: 1.8991
Epoch 2/10
2044/2044 ━━━━━━━━━━━━━━━━━━━━ 3s 1ms/step - accuracy: 0.4604 - loss: 1.9391 - val_ac
curacy: 0.4632 - val_loss: 1.8852
Epoch 3/10
2044/2044 ━━━━━━━━━━━━━━━━━━━━ 3s 1ms/step - accuracy: 0.4645 - loss: 1.9167 - val_ac
curacy: 0.4641 - val_loss: 1.8784
Epoch 4/10
```

```
2044/2044 ━━━━━━━━━━━━━━━━━━━━ 3s 1ms/step - accuracy: 0.4650 - loss: 1.9054 - val_ac
curacy: 0.4648 - val_loss: 1.8756
Epoch 5/10
2044/2044 ━━━━━━━━━━━━━━━━━━━━ 3s 1ms/step - accuracy: 0.4602 - loss: 1.9099 - val_ac
curacy: 0.4653 - val_loss: 1.8724
Epoch 6/10
2044/2044 ━━━━━━━━━━━━━━━━━━━━ 3s 1ms/step - accuracy: 0.4641 - loss: 1.9003 - val_ac
curacy: 0.4658 - val_loss: 1.8713
Epoch 7/10
2044/2044 ━━━━━━━━━━━━━━━━━━━━ 3s 1ms/step - accuracy: 0.4674 - loss: 1.8910 - val_ac
curacy: 0.4644 - val_loss: 1.8689
Epoch 8/10
2044/2044 ━━━━━━━━━━━━━━━━━━━━ 3s 1ms/step - accuracy: 0.4636 - loss: 1.8956 - val_ac
curacy: 0.4654 - val_loss: 1.8690
Epoch 9/10
2044/2044 ━━━━━━━━━━━━━━━━━━━━ 3s 1ms/step - accuracy: 0.4665 - loss: 1.8893 - val_ac
curacy: 0.4650 - val_loss: 1.8671
Epoch 10/10
2044/2044 ━━━━━━━━━━━━━━━━━━━━ 3s 1ms/step - accuracy: 0.4658 - loss: 1.8865 - val_ac
curacy: 0.4658 - val_loss: 1.8662
3220/3220 ━━━━━━━━━━━━━━━━━━━━ 2s 616us/step - accuracy: 0.4660 - loss: 1.8708
7513/7513 ━━━━━━━━━━━━━━━━━━━━ 4s 571us/step - accuracy: 0.4719 - loss: 1.8345
Test accuracy: 0.4664550721645355
Train accuracy: 0.47081458568573
```

In [ ]:

```python
# Illustrating training accuracy score of our three models using 80% of the data
models = ['Decision Tree', 'Random Forest', 'Neural Network']

train_acc_2 = [train_dt_accuracy_2, train_rf_accuracy_2, train_acc_2]
test_acc_2 = [test_dt_accuracy_2,  test_rf_accuracy_2,  test_acc_2]

x = np.arange(len(models))
width = 0.35

plt.figure(figsize=(8,5))

plt.bar(x - width/2, train_acc_2, width, label='Second Train Accuracy')
plt.bar(x + width/2, test_acc_2,  width, label='Second Test Accuracy')

plt.xticks(x, models)
plt.ylabel('Accuracy')
plt.title('Model Accuracy Comparison')
plt.legend()

plt.show()
```



Adding sample size from 50% to 80% of the total data doesn't seem to improve the accuracy of the model. This could also be due to the large number of columns we are working with. We will now attempt to run our algorithms but after reclassifying our columns into the a few categories of factors we predicted would affect accident crashes. These were:

1. Environmental
2. Vehicle Defect
3. Impairment/Distraction
4. Unknown
5. Driver Error

5.7.5 Classifying the causes into 5 groups and using pipelines

In [58]:

```
# We will run this using merged_df_2 which contains 80% of the sample
# Group the Target Variable
def group_cause(cause):
    if 'WEATHER' in cause or 'VISION OBSCURED' in cause:
        return 'Environmental'
    elif 'EQUIPMENT' in cause or 'DEFECTS' in cause:
        return 'Vehicle Defect'
    elif 'IMPAIRED' in cause or 'DISTRACTION' in cause:
        return 'Impairment/Distraction'
    elif 'UNABLE TO DETERMINE' in cause or 'NOT APPLICABLE' in cause:
        return 'Unknown'
    else:
        return 'Driver Error'

merged_df_2['BROAD_CAUSE'] = merged_df_2['PRIM_CONTRIBUTORY_CAUSE'].apply(group_caus

# Drop rows where the cause is 'Unknown'
merged_df_2 = merged_df_2[merged_df_2['BROAD_CAUSE'] != 'Unknown'].copy()

# Define Features (X) and Target (y)
```

```python
target_column = "BROAD_CAUSE"
# Drop the target and the original specific cause to prevent data leakage
X = merged_df_2.drop(columns=[target_column, 'PRIM_CONTRIBUTORY_CAUSE'])
y = merged_df_2[target_column]

# Column types for Preprocessing
categorical_cols = X.select_dtypes(include=["object", "category"]).columns
numerical_cols = X.select_dtypes(include=["int64", "float64"]).columns

# Preprocessing pipeline
preprocessor = ColumnTransformer(transformers=[
    ("num", StandardScaler(), numerical_cols),
    ("cat", OneHotEncoder(handle_unknown="ignore"), categorical_cols)
])

# train-test split (70/30)
# stratify=y ensures the class imbalance is equal in both train and test sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)

# Unified pipeline builder
def build_pipeline(model):
    return Pipeline(steps=[
        ("preprocessor", preprocessor),
        ("model", model)
    ])

# Define interpretable models
pipelines = {
    # Added class_weight='balanced' to force the models to look at rare crashes
    "best_DT": build_pipeline(DecisionTreeClassifier(max_depth=10, class_weight="bal

    # n_jobs=-1 tells your Mac to use all cores to train this massive forest faster
    "random_forest": build_pipeline(RandomForestClassifier(n_estimators=50, max_dept
    "neural_network": build_pipeline(MLPClassifier(hidden_layer_sizes=(100,), max_it
}

# Train and Evaluate
results = {}

for name, pipe in pipelines.items():
    print(f"--> Training {name}...")
    pipe.fit(X_train, y_train)

    y_train_pred = pipe.predict(X_train)
    y_test_pred = pipe.predict(X_test)

    train_acc = accuracy_score(y_train, y_train_pred)
    test_acc = accuracy_score(y_test, y_test_pred)

    results[name] = (train_acc, test_acc)

    # Save model for Tableau/Web App
    joblib.dump(pipe, f"{name}_80_percent.pkl")

    # Print the detailed classification report
    print(f"\n--- {name} Classification Report ---")
    print(classification_report(y_test, y_test_pred))

# Printing Results
print("\n=== FINAL ACCURACY SCORES ===")
for name, (train_acc, test_acc) in results.items():
    print(f"{name}")
```

```
    print(f"  Train Accuracy: {train_acc:.4f}")
    print(f"  Test Accuracy : {test_acc:.4f}")
```

```
--> Training best_DT...

--- best_DT Classification Report ---
                        precision    recall  f1-score   support

          Driver Error       0.98      0.54      0.70     59976
         Environmental       0.16      0.73      0.26      2053
Impairment/Distraction       0.03      0.51      0.06      1188
        Vehicle Defect       0.12      0.31      0.18       708

              accuracy                           0.54     63925
             macro avg       0.32      0.52      0.30     63925
          weighted avg       0.92      0.54      0.66     63925

--> Training random_forest...

--- random_forest Classification Report ---
                        precision    recall  f1-score   support

          Driver Error       0.98      0.70      0.82     59976
         Environmental       0.18      0.73      0.29      2053
Impairment/Distraction       0.06      0.35      0.10      1188
        Vehicle Defect       0.07      0.50      0.12       708

              accuracy                           0.70     63925
             macro avg       0.32      0.57      0.33     63925
          weighted avg       0.92      0.70      0.78     63925

--> Training neural_network...

--- neural_network Classification Report ---
                        precision    recall  f1-score   support

          Driver Error       0.95      0.99      0.97     59976
         Environmental       0.43      0.27      0.33      2053
Impairment/Distraction       0.23      0.03      0.06      1188
        Vehicle Defect       0.53      0.18      0.27       708

              accuracy                           0.94     63925
             macro avg       0.53      0.37      0.41     63925
          weighted avg       0.91      0.94      0.92     63925


=== FINAL ACCURACY SCORES ===
best_DT
  Train Accuracy: 0.5503
  Test Accuracy : 0.5442
random_forest
  Train Accuracy: 0.7126
  Test Accuracy : 0.6966
neural_network
  Train Accuracy: 0.9594
  Test Accuracy : 0.9356
```

From the above analysis, we conclude that regrouping our columns into factors yielded better accuracy scores. These were our outcomes:

1. For Decision Trees, training accuracy was 56% and testing accuracy was also 56%
2. For Random Forest Classifier, training accuracy was 68% and testing accuracy was 67%
3. For Neural Networks, training accuracy was 95% and testing accuracy was 93%

- Therefore our best model based on highest training and testing accuracy was neural networks.

6.0 Recommendations

From our project:

1. The most primary contributory cause of accidents was failing to yield right of way.
2. Human, vehicle, environmental and temporal characteristics all contributed significantly to accident crashes.
3. We can safely predict with the above characteristics, the primary contributory factor to accident crashes with a 93% accuracy using trained neural networks (cars_accidents_project/random_forest_80_percent.pkl).

7.0 Limitations

1. Computational Resource Constraints: The original dataset is massive, containing over 850,000 distinct entries. Training complex machine learning algorithms (such as Neural Networks and Random Forests) on standard local hardware (e.g., machines with 16 GB of RAM) resulted in severe memory exhaustion and computationally prohibitive training times. Consequently, to balance computational feasibility with model performance, the final models were trained on an 80% representative sample of the dataset rather than the entirety of the data.

2. Dataset Granularity and Passenger Exclusion: The raw dataset is recorded at the "person" level rather than the "crash" level, meaning every individual involved (including passengers) generates a distinct row. To prevent artificial inflation of the data and to focus our insights strictly on the active decision-makers on the road, we filtered the dataset to analyze only drivers. While this was a necessary step, it means our findings do not account for passenger demographics or casualty rates.

3. Multi-Vehicle Incident Weighting: Because our analysis was conducted at the driver level, a single collision involving two or more drivers results in multiple rows sharing the same crash report. This inherent hierarchical structure means that multi-vehicle crashes carry a proportionally heavier weight in our predictive models than single-vehicle crashes, which introduces a slight structural bias when analyzing overall primary contributory causes.

8.0 Suggestions for Further Studies

1. Cloud Computing Integration: Future iterations of this project should utilize cloud-based computing platforms (such as AWS EC2 instances or Google Colab Pro) and distributed data processing frameworks like PySpark. This would provide the necessary computational power to train complex Neural Networks on the full 100% dataset without memory bottlenecks.

2. Crash-Level Aggregation: To resolve the multi-vehicle weighting limitation, future studies could involve aggregating the dataset up to the Crash Level (grouping by CRASH_RECORD_ID). This would require advanced feature engineering to summarize the demographics of all drivers involved in a single incident, allowing for a pure 1:1 prediction of crash causes.